

32548-48730, Cybersecurity

Week-5

Operating System Security



References:

Stallings, Brown; Computer Security, 4th edition,
Chapter 10, Wenliang Du; Computer Security, Chapter 4, Chapter 8

A/Prof. Priyadarsi Nanda
Priyadarsi.Nanda@uts.edu.au

Objectives



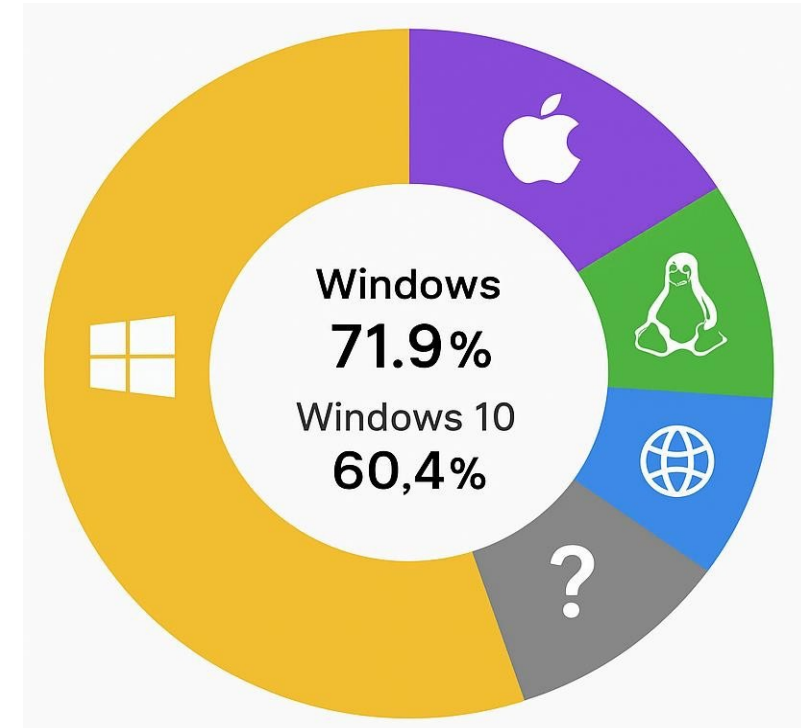
- OS security, also, called System security
 - Steps to secure a system
 - An important aspect in Cybersecurity
- Buffer overflow attack and consequences
- Race condition and Dirty-COW attack
- Mobile operating system security
- Review Questions:
 1. Why is system security considered important in Cybersecurity?
 2. Discuss the basic steps required to secure operating system?
 3. What measures are taken to defend against buffer overflow attack?
 4. What penetration testing tools are available for mobile operating systems?

Leading Desktop Operating Systems Worldwide



Key Market Share Stats (StatCounter Global Stats)

- **Windows – 71.9%**
 - Windows 11 ≈ 36.6%
 - Windows 10 ≈ 60.4% (but declining with end-of-support coming Oct 2025)
- **macOS – 15.0%** (steady adoption, slow growth)
- **Linux – 4.0–4.7%** (gaining traction among developers, education, and enterprises)
- **Chrome OS – 1.9%** (dominant in education & budget laptops)
- **Others/Unknown – ~6–7%** (niche systems & unclassified OS reports)



<https://gs.statcounter.com/os-marketshare/desktop/worldwide/%E2%80%9D>

Operating Systems in Cybersecurity – Most Widely Used



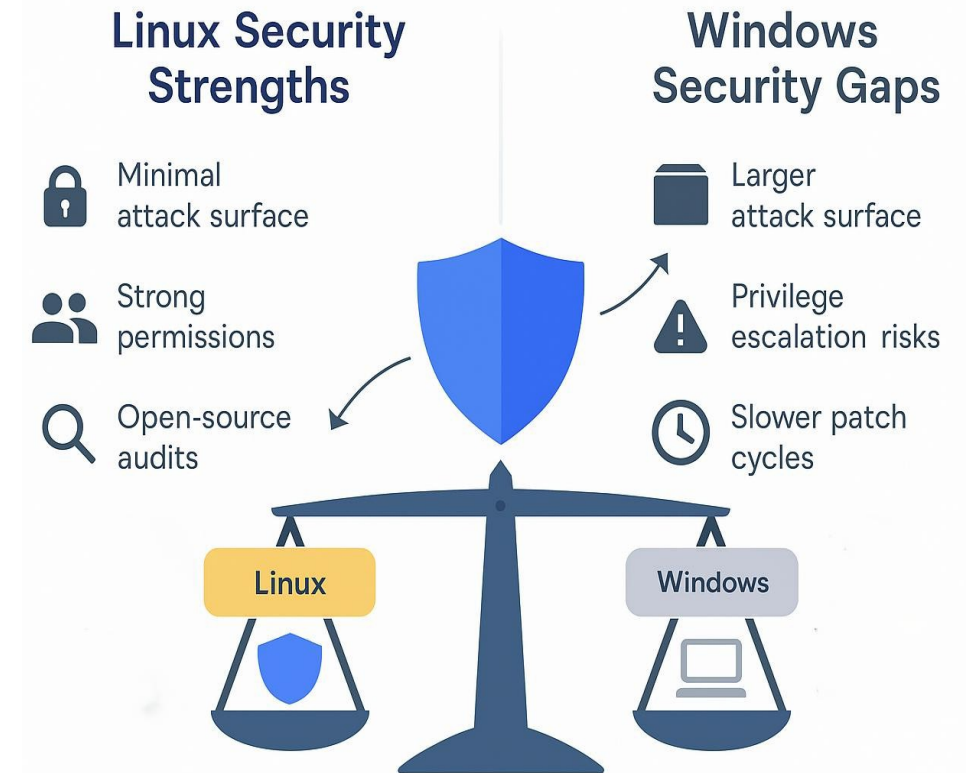
- **Linux (esp. Kali Linux)** – Dominates in cybersecurity. Preferred by professionals for:
 - Penetration testing
 - Digital forensics
 - Reverse engineering
 - Wide range of pre-installed security tools
- **Windows** – Still essential in enterprise environments due to ubiquity. Many malware analyses, detection tools, and reverse engineering efforts rely on Windows.
- **macOS** – Moderate adoption, especially among researchers using macOS-specific hardware or tools.
- **Security-focused Linux Distros** (tailored for cyber defense):
 - **Kali Linux** – Preloaded with pen-testing & threat analysis tools.
 - **Qubes OS** – High isolation via virtualization; strong compartmentalization for secure workflows.
 - **Tails & Whonix** – Built for anonymity, privacy, and secure, live-mode usage without leaving traces.
- **Termux**- Penetration Testing on Android
- **iSH Shell**- Penetration Testing on iOS

<https://www.techradar.com/news/best-linux-distro-privacy-security>

Focus on Linux in Cybersecurity



- **Minimal attack surface:** Linux distributions can be stripped down to essential components, reducing exploitable code.
- **Granular permissions:** Strong file ownership, group policies, and privilege separation limit the blast radius of attacks.
- **Open-source transparency:** Global community constantly audits and patches vulnerabilities faster than many proprietary OSs.
- **System hardening:** Security modules like **SELinux** and **AppArmor** enforce strict access controls.
- **Specialized distros:** Security-focused Linux builds (Kali, Qubes, Tails) provide dedicated penetration testing, isolation, or anonymity features.
- **Lower malware prevalence:** Compared to Windows, Linux has significantly fewer widespread malware families.
- **Community-driven security:** Frequent updates, patches, and dedicated security mailing lists enhance resilience.



Linux vs Windows in Cybersecurity: Case Studies



1. Linux Case Study: Defense & Government Security (SELinux)

- **System:** SELinux deployed by the U.S. Department of Defense.
- **Outcome:** Mandatory Access Controls (MAC) prevent privilege escalation even when apps are compromised.

Linux can enforce *military-grade compartmentalization of sensitive data*.

3. Linux Case Study: Android Ecosystem (Linux Kernel)

- **System:** Billions of Android devices protected by Linux kernel security + SELinux policies.
- **Outcome:** Despite huge attack surface, Linux kernel sandboxing limits systemic compromise.

Linux scales securely from **servers to mobile**.

5. Linux Case Study: Cloud & Hyperscale (Google, AWS, Meta)

- **System:** Nearly all hyperscale data centres run hardened Linux servers.
- **Example:** Google patched its Linux fleet quickly during the **Heartbleed OpenSSL crisis (2014)**.
Open-source ecosystems allow **rapid detection + patching**, reducing exposure windows.

2. Windows Case Study: WannaCry Ransomware (2017)

- **System:** Exploited Windows SMBv1 vulnerability (*EternalBlue*).
- **Impact:**
 - Infected **over 200,000 computers in 150+ countries**.
 - Critical services like the **UK's NHS** were paralyzed.
- **Root Cause:** Lack of patching, centralized reliance on Windows in critical infrastructure.
Windows' ubiquity + slow patch adoption = massive attack surface.

4. Windows Case Study: Stuxnet Worm (2010)

- **System:** Targeted **Windows-based SCADA systems** controlling Iranian nuclear centrifuges.
- **Impact:**

- Caused physical damage to centrifuges.
- First cyberweapon that bridged *digital* → *physical*.

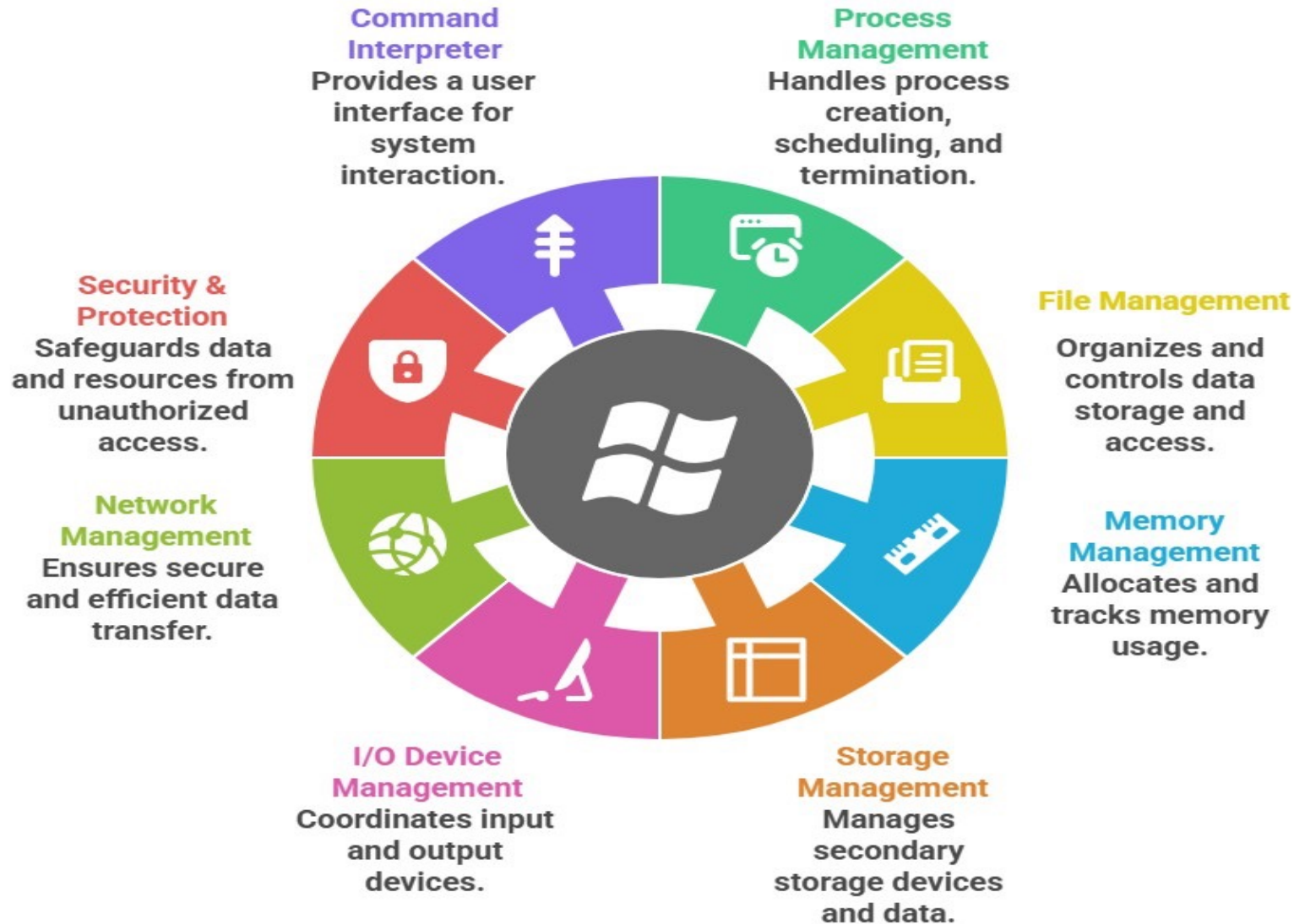
This Shows how **Windows systems embedded in critical infrastructure can become a national security risk.**

6. Windows Case Study: Exchange Server Hacks (2021)

- **System:** Microsoft Exchange Servers exploited by **Hafnium APT** group.
- **Impact:**
 - Hundreds of thousands of organizations compromised.
 - Attackers planted persistent backdoors.

Proprietary systems often have **slow vulnerability disclosures** and **slower patch rollout**.

Desktop Operating System



<https://www.javatpoint.com/components-of-operating-system>

Cyber Security, Week-5, OS security

Desktop Operating System Security



Security across OS layers:

•Hardware Level

- Physical device protection
- Access control to hardware resources

•OS Kernel

- Hardware interaction and resource allocation
- Enforces **access control** and **authentication functions**

•Operating System (OS)

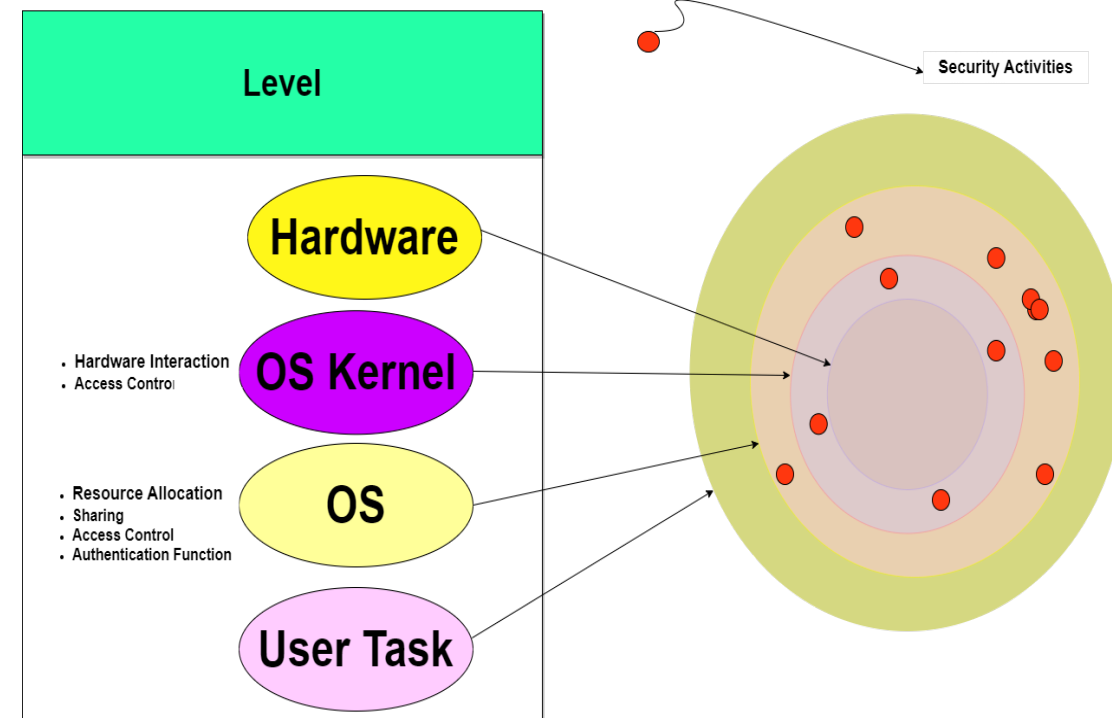
- Manages sharing of resources
- Implements **user permissions** and security policies

•User Task Layer

- Applications and user processes
- Security depends on **permissions, isolation, and monitoring**

Key Point:

Security activities span all layers, from hardware up to user applications. Weakness at any level can expose the entire system at risk.



Desktop Operating System Security



- **Vulnerability During Installation**

A system may be exposed to threats during installation **before patches are applied**. Attackers actively look for unpatched vulnerabilities.

- **Need for a Planned Security Process**

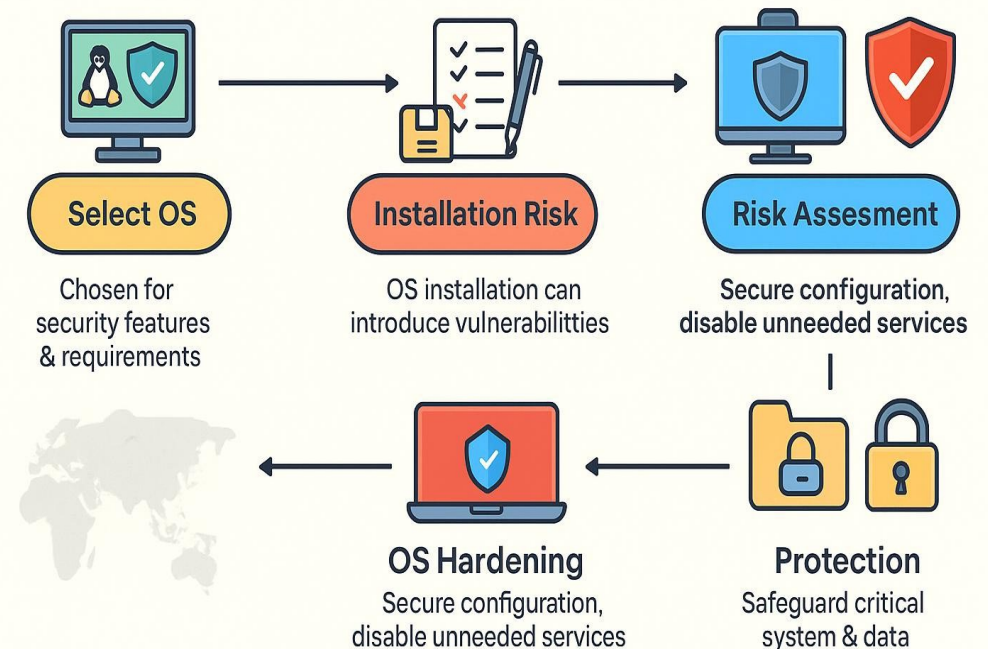
To secure systems against threats, organizations must:

- **Assess risks** and design a secure deployment strategy.
- **Secure the operating system** first, then critical applications.
- Ensure **critical system files and configurations** are protected.
- Implement **network protection mechanisms** (e.g., firewalls, IDS/IPS).
- Apply **continuous monitoring and maintenance processes**.

Key Takeaway:

OS security requires **structured, layered defense**: from installation through operation to reduce risk of compromise.

Secure OS Deployment Process



Operating System Hardening Benchmarks



Why Benchmarks?

- Provide consistent, proven security configurations.
- Reduce misconfigurations and improve compliance.

Examples:

- **CIS Benchmarks** – Practical step-by-step secure configs for OS, apps, and cloud.
- **DISA STIGs** – Defense-grade security standards, detailed and strict.
- **NIST Guidelines** – Broader frameworks (e.g., NIST 800-53) for compliance.
- **Vendor Guides** – Microsoft, Red Hat, Apple publish OS-specific hardening advice.

Key Point:

Adopting these benchmarks lowers attack surfaces and ensures systems meet both practical and compliance needs.



Designing a Secured system



🔑 Authentication

- **Purpose:** Verifies user identity and permits access to the operating system.

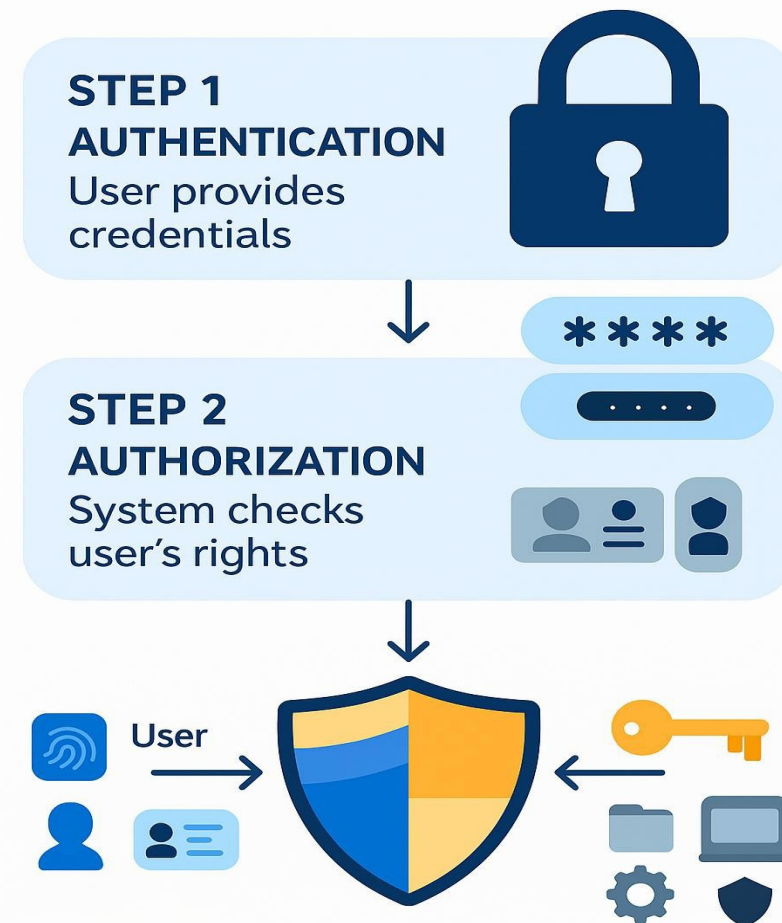
Who you are?

- **Physical authentication:**
 - Controls physical entrance (e.g., office/property access).
 - Uses **magnetic cards** or **biometric measures** (fingerprint, retina scan).
- **Digital authentication:**
 - Confirms identity through digital credentials (passwords, smart cards, MFA tokens).

🔑 Authorization

- **Purpose:** Determines what authenticated users are allowed to do. What resources you are allowed?

- **Process:**
 - Performs **after authentication** is successful.
 - Decides which functions/resources the user can access.
 - Deals with **privileges, rights, and access levels**.



Secured System Best Practices



User Administration:

- **Account Creation:** Ensure every user has a unique ID linked to their role.
- **Password Policies:** Enforce length (≥ 12 characters), complexity (uppercase, lowercase, digits, symbols), and expiration (e.g., 90 days).
- **Privilege Management:** Apply the *principle of least privilege* (users only get what they need to do their job).
- **Lifecycle Management:** Accounts should be regularly reviewed and disabled immediately upon role change or exit.

Best Practices:

- **Consistent Naming Convention:** Makes auditing and automation easier (e.g., [first.last@domain](#)).
- **Mandatory First Login Reset:** Temporary passwords must be reset at first use.
- **Password Protection:** Store only hashed + salted passwords, never plain text.
- **Avoid Defaults:** Never keep default usernames/passwords active.
- **Regular Auditing:** Periodic review to identify unused, dormant, or high-risk accounts.
- **Multi-Factor Authentication (MFA):** Strongly recommended for privileged accounts.

Trusted Operating System



•Trusted Computing Base (TCB):

The combination of hardware, software, and controls that are essential for enforcing a system's security policy.

- If the TCB fails, the entire system's security fails.

• Acts as the **foundation of trust** for the entire system.

Components

1. Hardware

1. Physical devices and circuitry that support secure processing.

2. Security Kernel

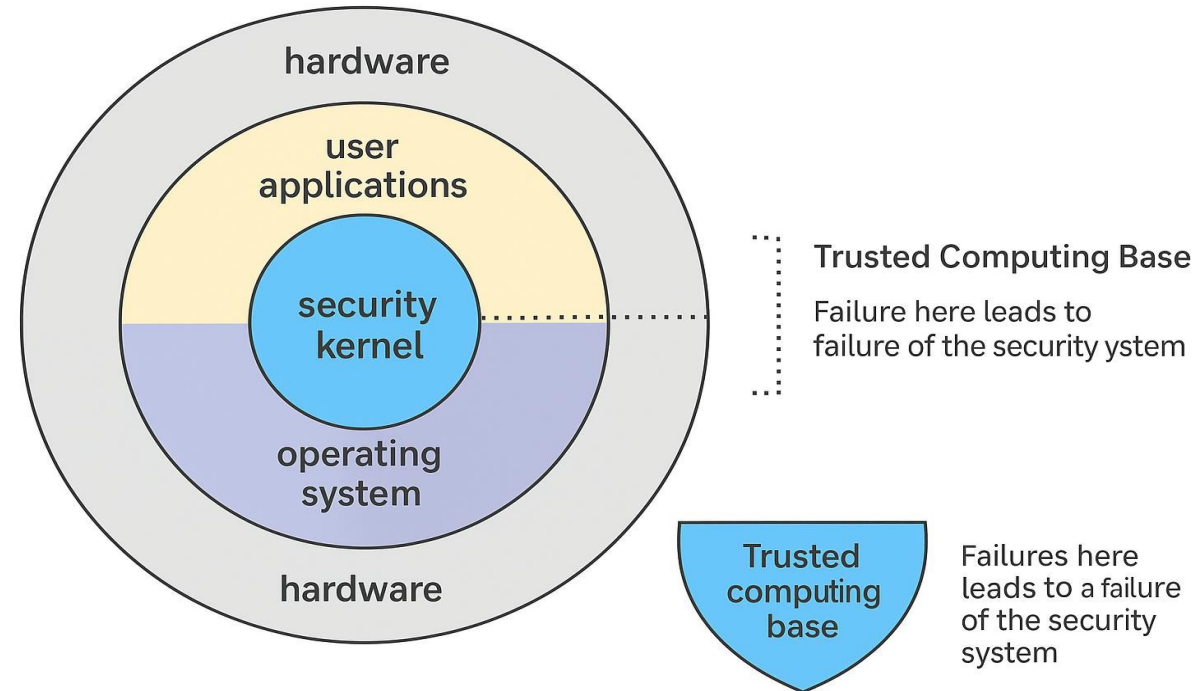
1. Core part of the OS that enforces strict access control.
2. Mediates all interactions between subjects (users/programs) and objects (data/resources).

3. Operating System

1. Provides services, process isolation, and manages security policies.

4. User Applications

1. Rely on the secure foundation provided by underlying OS layers.



Even with Trusted Operating Systems and strong security foundations, human mistakes and software errors often open the door to attacks.

Mistakes start with errors!!!



1. Poor Design Choices / Assumptions

- **Incorrect Privilege Separation** → giving applications *administrator/root* access unnecessarily. (Common in early Windows versions where users ran as “admin” by default → huge attack surface).
- **Weak Default Settings** → “open” permissions, default passwords (**admin/admin**), or services enabled by default.
Real-world case: Mirai botnet spread by exploiting default IoT passwords.
- **Poor Failure Modes** → systems should fail *securely*, not *openly*. If login server fails, access should be denied, not automatically granted.
- **Lack of Review** → skipping security design reviews leads to missed vulnerabilities (e.g., Heartbleed in OpenSSL went unnoticed for 2+ years due to poor code audit).

2. Programming Errors

- **Buffer Overflows** → Classic exploit: Worms like Code Red & Slammer used buffer overflow vulnerabilities to spread globally in minutes. Modern mitigations (ASLR, DEP) exist, but poorly written C/C++ code is still vulnerable.
- **Race Conditions** → Example: TOCTOU (Time-Of-Check-Time-Of-Use) bugs where file permissions are checked, but attacker swaps files before execution.

3. Awareness & Human Factors

- **Insufficient Testing** → Many breaches come from bugs missed due to limited QA security testing. Security testing must go beyond functionality.
- **Bad or Unsafe Languages** → Legacy reliance on C for performance leaves many systems open to memory corruption bugs.
- **Predictable Randomness** → Weak random number generators make crypto insecure (e.g., Debian’s infamous OpenSSL RNG bug, where all keys generated were predictable for 2 years).
- **Legacy Code** → Old codebases often contain outdated, insecure functions (e.g., **strcpy, gets**).
- **Poor Documentation** → Developers misconfigure security features because documentation is vague or outdated (example: developers disabling TLS verification due to unclear docs).

Memory and File system Security



Core Concept:

- A computer's **contents are encapsulated in its memory and file system.**
- Protecting memory and file systems is **fundamental for securing overall system integrity.**

Why it matters:

- Attackers often target memory (buffer overflows, injection) or file systems (malware, unauthorized access).
- Protection mechanisms must adapt to **different architectures** (16-bit, 32-bit, 64-bit, little-endian, big-endian).

Security Focus Areas:

- **Memory Protection:** Prevent unauthorized access to process and data in RAM.
- **File System Security:** Enforce access controls, encryption, and integrity checks.
- **Process Isolation:** Ensure programs cannot interfere with each other's memory space.

👉 *System protection begins with securing memory and file systems, as they hold the most critical system and user data.*

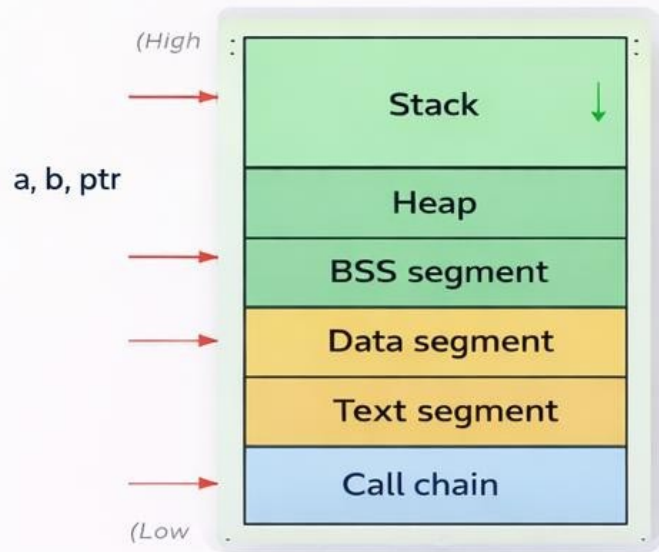
Memory Stack in Modern Security



Understanding stack layout is still essential because modern attacks and defenses both depend on how function calls, return addresses, and local memory are organized.

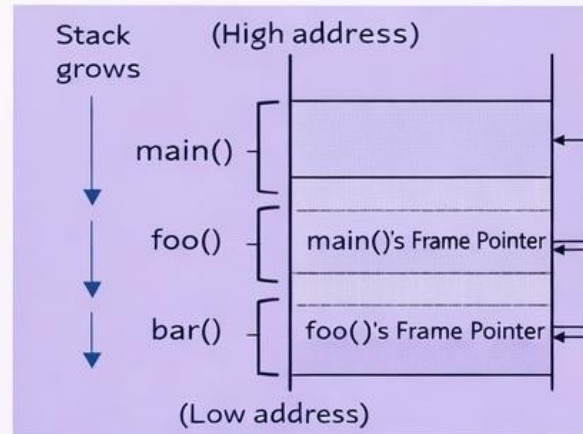
Memory Layout Basics

- Stack, heap, data, BSS, text



Function Call Stack

- Arguments, local variables
- Frame pointer, return address, call chain



- Arguments, local variables
- Frame pointer, return address, call chain

Security Relevance Today

-  **Buffer overflow**, return-address-overwrite, ROP
-  **Stack canaries** detect stack smashing
-  **ASLR** randomizes memory locations
-  **DEP/NX bit** prevents executing on the stack
-  **Control-Flow Integrity (CFI)**, **Shadow Stack**
-  **Memory-safe languages like Rust**
-  **Secure coding practices** (bounds check, APIs)

Memory layout: Program memory is divided into code, global data, heap, and stack.

Function call stack: Each function call creates a stack frame containing arguments, local variables, and the return address.

Vulnerable Program



```
/* stack.c */  
  
/* This program has a buffer overflow vulnerability. */  
  
#include <stdlib.h>  
#include <stdio.h>  
#include <string.h>  
  
int foo(char *str)  
{  
    char buffer[100];  
  
    /* The following statement has a buffer overflow problem */  
    strcpy(buffer, str);  
  
    return 1;  
}
```

Why It's Dangerous:

strcpy() does not validate input length.

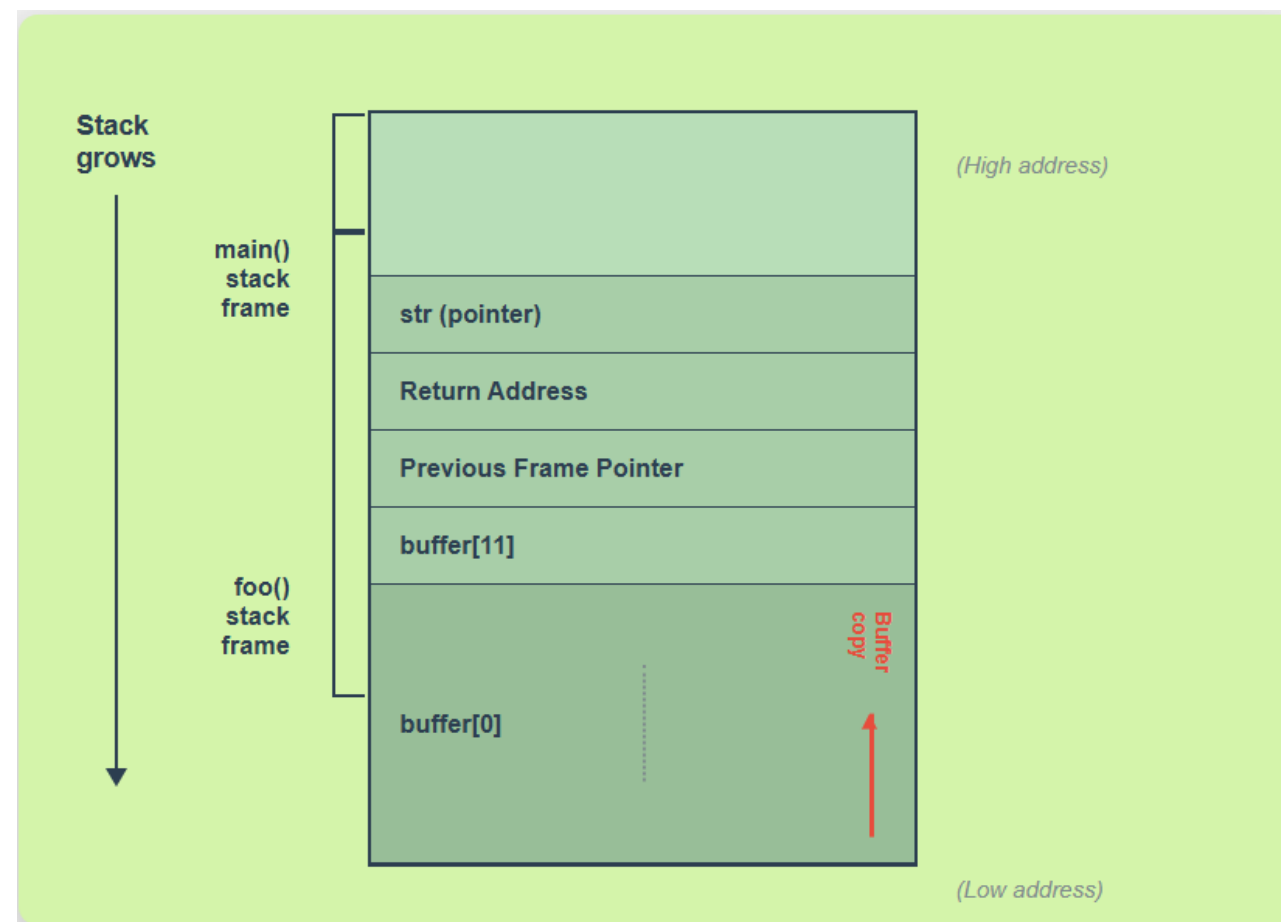
If str is larger than 100 bit, it overwrites memory beyond the buffer. Attackers can overwrite the **return address**, hijacking control flow.

Unchecked input → Buffer Overflow → Possible System Compromise.

Use secure coding practices and memory-safe functions to prevent exploitation.

Buffer Overflow occurs when data written to a buffer exceeding its allocated size, overwriting adjacent memory.

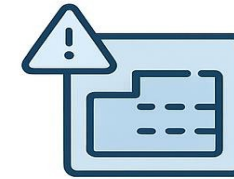
- This can corrupt data, crash programs, or allow attackers to inject and execute malicious code.



Consequences

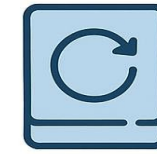


- **Consequences of Buffer Overflow Exploits**
 - **Overwriting return address with a random value causing:**
 - ❌ **Invalid instruction** → program executes garbage (could be malicious code) and crashes.
 - ❌ **Non-existing address** → system raises segmentation fault.
 - ❌ **Access violation** → illegal access to protected memory.
- **If attacker controls the overwritten address:**
 - ⚠️ **Code Injection Attack**
 - Attacker places **malicious code** in memory.
 - Return address is overwritten to point to this malicious code.
 - Program unknowingly executes attacker's code.
- **Resulting impact:**
 - Unauthorized access to system resources.
 - Privilege escalation (attacker becomes admin/root).
 - Full system compromise (data theft, service disruption, malware installation).



Buffer Overflow

Input exceeds buffer size → overwrites memory



Return Address Overwritten

Old return address replaced with attacker's value



Control Hijack

Execution jumps to attacker's injected code



Malicious Code Execution

Attacker's shell/malware runs with program's privileges



Consequences

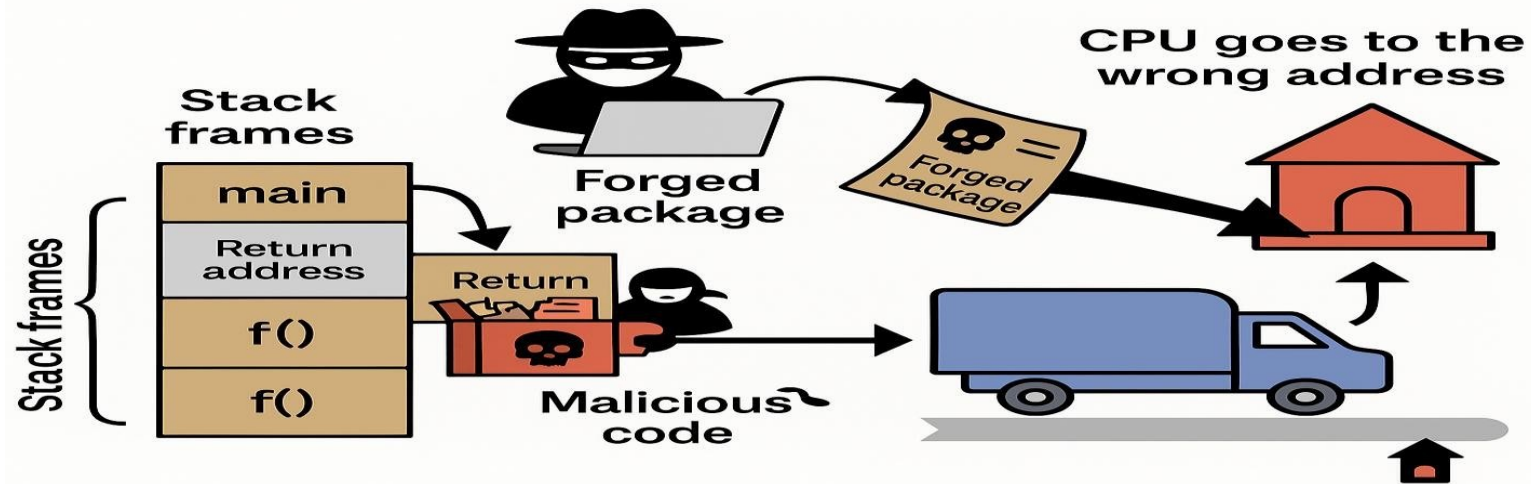
- Unauthorized system access
- Escalation to root/admin
- Data theft or system damage

Buffer Overflow



- A buffer overflow is a **classic and very common bug** in computer security.
- A **buffer** is simply an array where user-supplied data is stored.
- In some languages (notably **C and C++**), **arrays are not bounds-checked**.
- If input data is **too long**, it can **overflow the buffer** and overwrite adjacent memory.
- This can:
 - Corrupt valid data
 - Crash the program
 - Or allow an **attacker to take control** by inserting malicious code.

✓ **Key Point:** Buffer overflows turn programmer mistakes into **security vulnerabilities**.



Attacker's creation of Malicious Input (badfile)

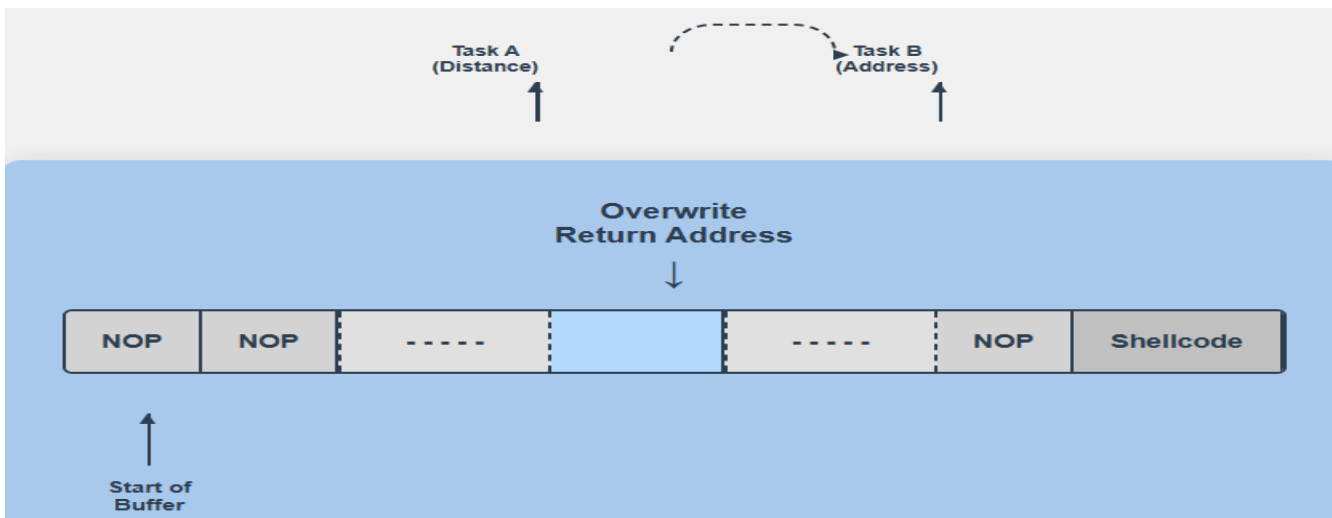


Task A:

Find the **offset distance** between the start of the buffer and the return address.

Task B:

Locate the **target memory address** to place the malicious shellcode.

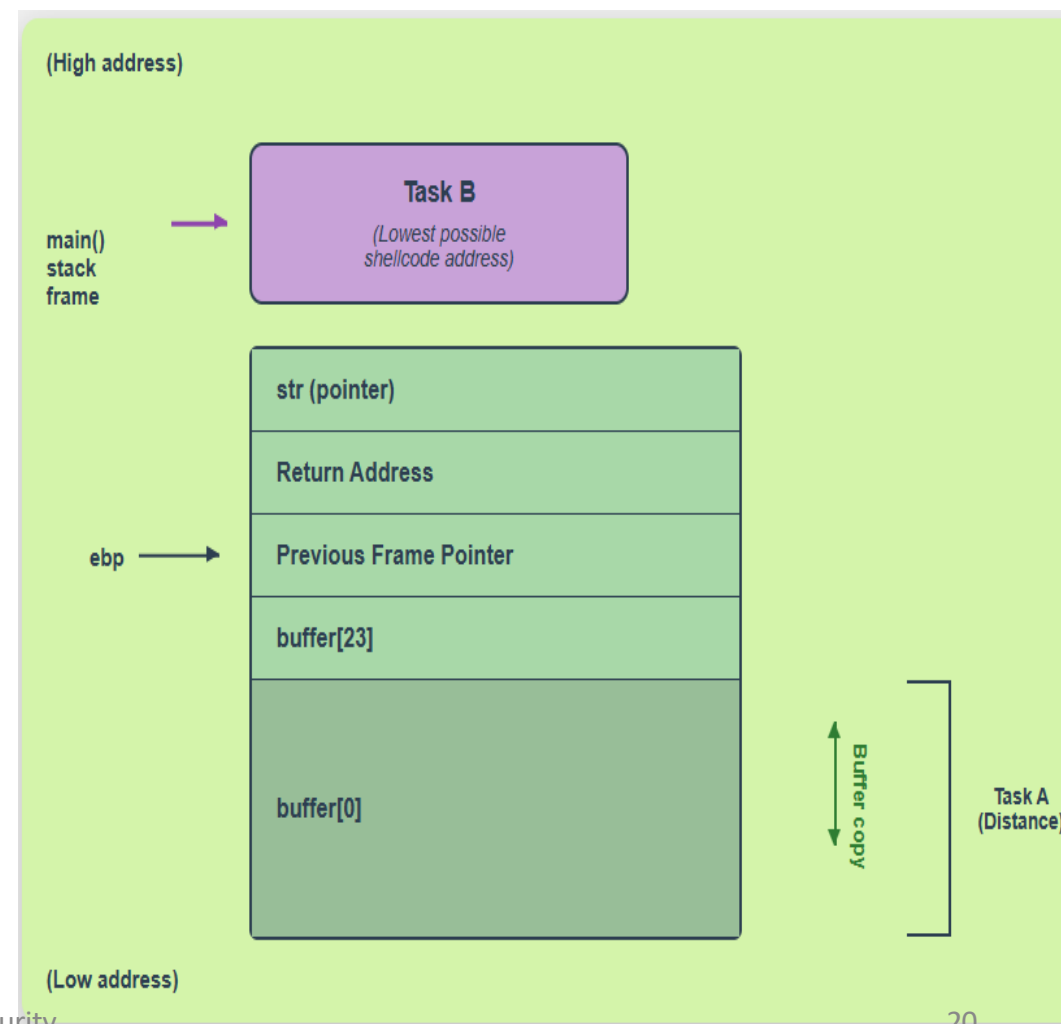


How the Attack Works

1. **Fill the Buffer** → Attackers provide oversized input that exceeds the buffer size.
2. **Overwrite Return Address** → The extra data overwrites the return address in the stack.
3. **Redirect Execution** → The overwritten return address points to the attacker's shellcode.
4. **Execute Malicious Code** → The shellcode is executed, giving attackers control.

(Attacker's Goal):

- “By carefully calculating **distance** and **address**, attackers hijack control flow.”



Use of GDB (GNU Debugger)?



- GDB (*GNU Debugger*) is a powerful command-line tool that helps programmers inspect and control program execution.
- It is widely used in C/C++ debugging.

Also, it allows you to:

- Step through code line-by-line
- View and modify memory, variables, and registers
- Set breakpoints to pause execution at critical spots
- Control program flow and observe behavior

❑ Why Attackers could use use GDB to launch Buffer Overflow Attacks

- Locate the **base address of the buffer** (**p &buffer**)
- View the **frame pointer** (**p \$ebp**) and calculate where, the **return address** lies
- Identify the exact **offset** needed to overwrite the return address
- Inspect memory or registers when the exploit occurs
- This hands-on visibility into program memory is essential for crafting precise exploits.

Common GDB Commands

\$ gcc -g -o prog prog.c	# Compile with debug symbols
\$ gdb ./prog	# Start debugging session
(gdb) b bof	# Set breakpoint at function 'bof'
(gdb) run	# Run program until breakpoint
(gdb) p &buffer	# Print buffer's base address
(gdb) p \$ebp	# Print frame pointer
(gdb) p \$ebp + 4 - [buffer_addr]	# Calculate offset to return address
(gdb) quit	# Exit GDB

<https://sourceware.org/gdb/documentation/>

Task A : Distance Between Buffer Base Address and Return Address



Finding the exact offset between the buffer's starting address and the return address, which is critical for crafting an exploit.

GDB Buffer Overflow Analysis Steps

1. Set breakpoint

```
(gdb) b bof
```

```
(gdb) run
```

2. Print buffer address

```
(gdb) p &buffer
```

3. Print frame pointer address

```
(gdb) p $ebp
```

4. Calculate distance

```
(gdb) p 0x02 - 0x01
```

5. Exit (quit)

```
(gdb) quit
```

- Breakpoint at vulnerable function using **gdb**
- Find the base address of buffer
- Find the address of the current frame pointer **(ebp)**
- Return address is **\$ebp + 4**

Why This Matters

- The calculated offset tells us **how much input is needed to overwrite the return address.**
- This is the key step before injecting shellcode or malicious instructions.
- Without this, the attacker cannot reliably redirect execution flow.

👉 **Result:** The calculated distance = number of bytes needed to overflow the buffer and overwrite the return address.

Task B : Address of Malicious Code



- Where malicious code (shellcode) resides in memory so that it can be executed.
 - Malicious code is written in a file (badfile) and passed as an argument to the vulnerable function.
 - Using the **GDB debugger**, attacker inspect the program at runtime to find the memory address of the function argument.
 - This allows attackers (or researchers simulating attacks) to determine where the injected code lives in memory.

•Steps:

1.Disable randomization (ASLR):

```
sudo sysctl -w kernel.randomize_va_space=0
```

2.Compile and run the program:

```
gcc prog.c -o prog
./prog
```

Output shows argument's address (e.g., 0xbffff370).

⚠ Without ASLR, the same code always runs at predictable addresses → making attacks easier.

```
#include <stdio.h>
void func(int al)
{
    printf(": al's address is 0x%x \n", (unsigned int) &al);
}
int main()
{
    int x = 3;
    func(x);
    return 1;
}
```

```
$ sudo sysctl -w kernel.randomize_va_space=0
kernel.randomize_va_space = 0
$ gcc prog.c -o prog
$ ./prog
: al's address is 0xbffff370
```

```
$ ./prog
: al's address is 0xbffff370
```

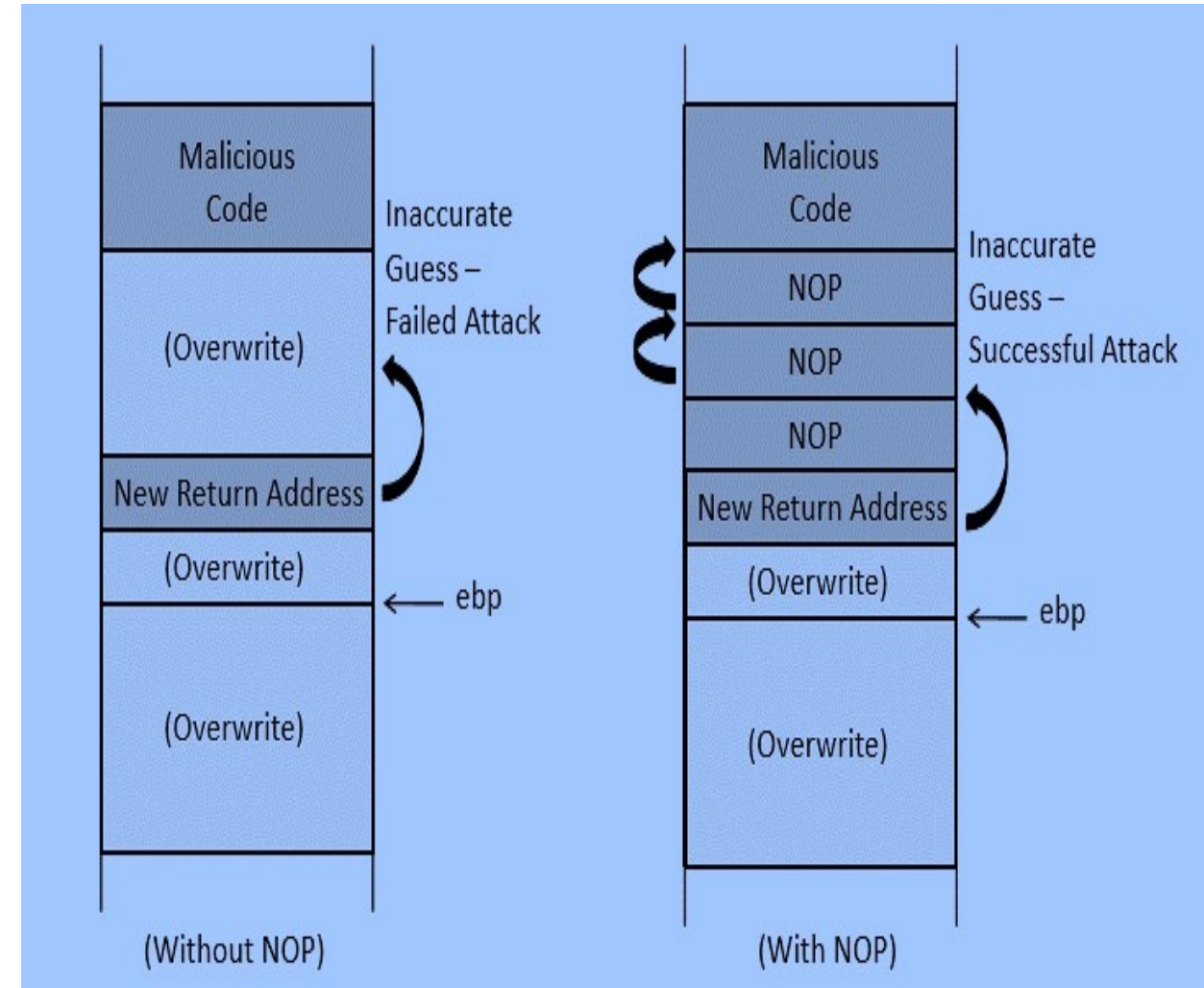
Using NOP to inject Malicious Code



- Finding exact location of malicious code address is often difficult because memory addresses can vary during execution.
- Instead, using **NOP sled** increases the chances of landing in the right spot.
 - **NOP (No Operation)**: An instruction that does nothing but moves execution forward.
 - Attackers fill the buffer with multiple NOP instructions followed by malicious code.
- Effect: Even if the guessed return address is inaccurate, execution will “slide” through the NOPs until it reaches the malicious code.

Why it works:

- Without NOPs → a small error in guessing the address **= failed attack.**
- With NOPs → errors in guessing still land inside the NOP sled, eventually reaching the malicious code **= successful attack.**



Execution Results



❑ Step 1: Compiling the vulnerable code

The program (stack.c) is compiled with **all security countermeasures disabled** to make it exploitable:

- **-z execstack**: Allows execution from the stack.
- **-fno-stack-protector**: Disables stack protection (canaries).
- **chmod 4755**: Sets the program as SUID root, giving root privileges when executed.

❑ Step 2: Compiling and running the exploit code

- Exploit program generates the **badfile** with malicious payload.
- Executing the vulnerable **stack** program with this badfile overwrites the return address.

❑ Step 3: Successful exploit

- The shellcode is executed, giving the attacker **root privileges** (**Got the root shell!**).

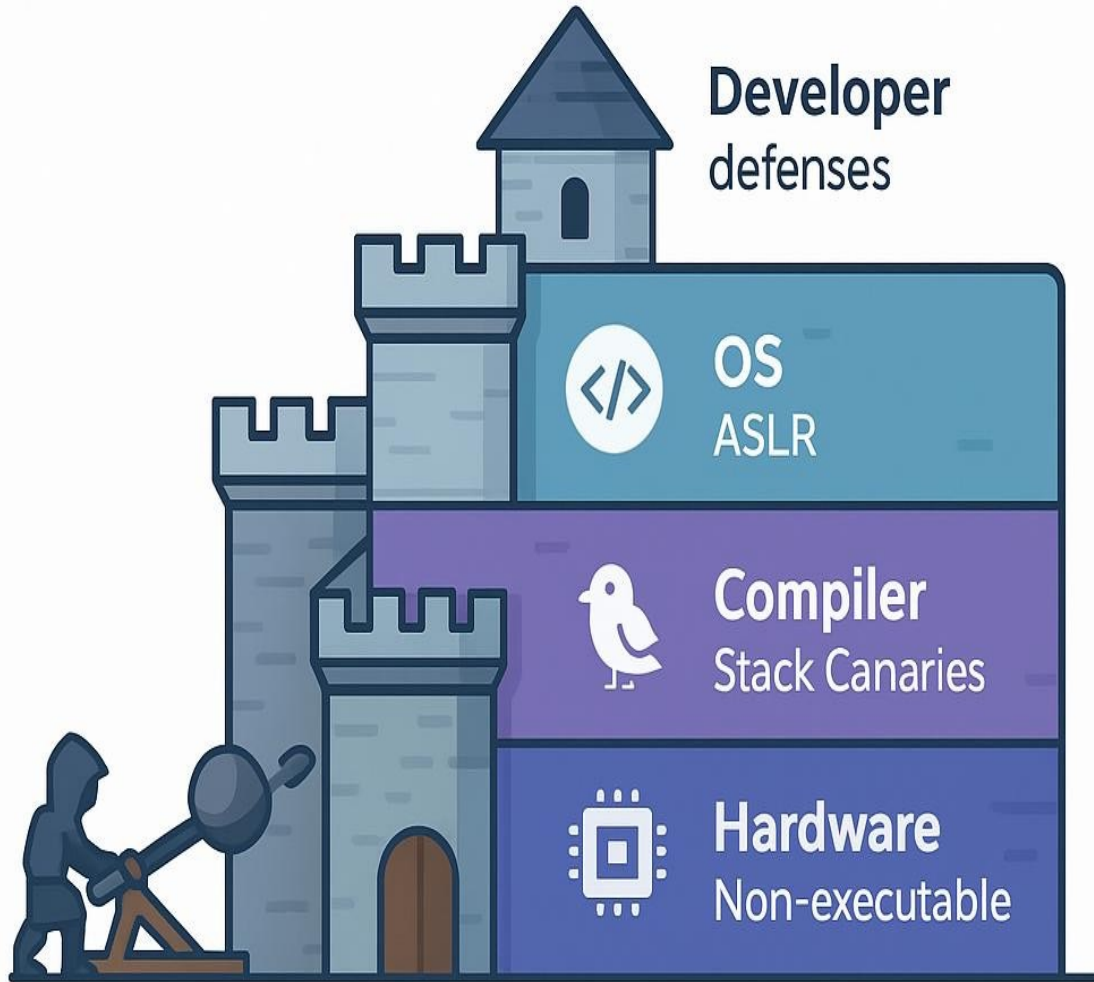
⚠ *This shows the real danger: without countermeasures, buffer overflows can directly lead to system compromise.*

```
$ gcc -o stack -z execstack -fno-stack-protector stack.c
$ sudo chown root stack
$ sudo chmod 4755 stack
```

```
$ gcc exploit.c -o exploit
$ ./exploit
$ ./stack
# id - Got the root shell!

uid=1000(seed) gid=1000(seed) euid=0(root)
groups=0(root), ...
```

Buffer Overflow: Countermeasures



❑ Developer Approaches

- Use **safe string functions** (e.g., `strncpy()`, `strncat()`) instead of unsafe ones (`strcpy`, `strcat`).
- Adopt **safe libraries** that automatically check input length.

➤ Example:

```
strncpy(dest, src, n); dest[n] = '\0';
```

❑ OS Approaches

- **ASLR (Address Space Layout Randomization):** Randomizes memory locations to make it harder for attackers to guess the position of code and data.

❑ Compiler Approaches

- **StackGuard / Stack Canaries:** Compiler inserts secret values before the return address; if altered, program aborts immediately.

❑ Hardware Approaches

- **Non-Executable Stack (NX bit / DEP):** Marks stack memory as non-executable so injected code cannot run.

Why We Need Countermeasures – and How Attackers Adapt



The Defender's Plan

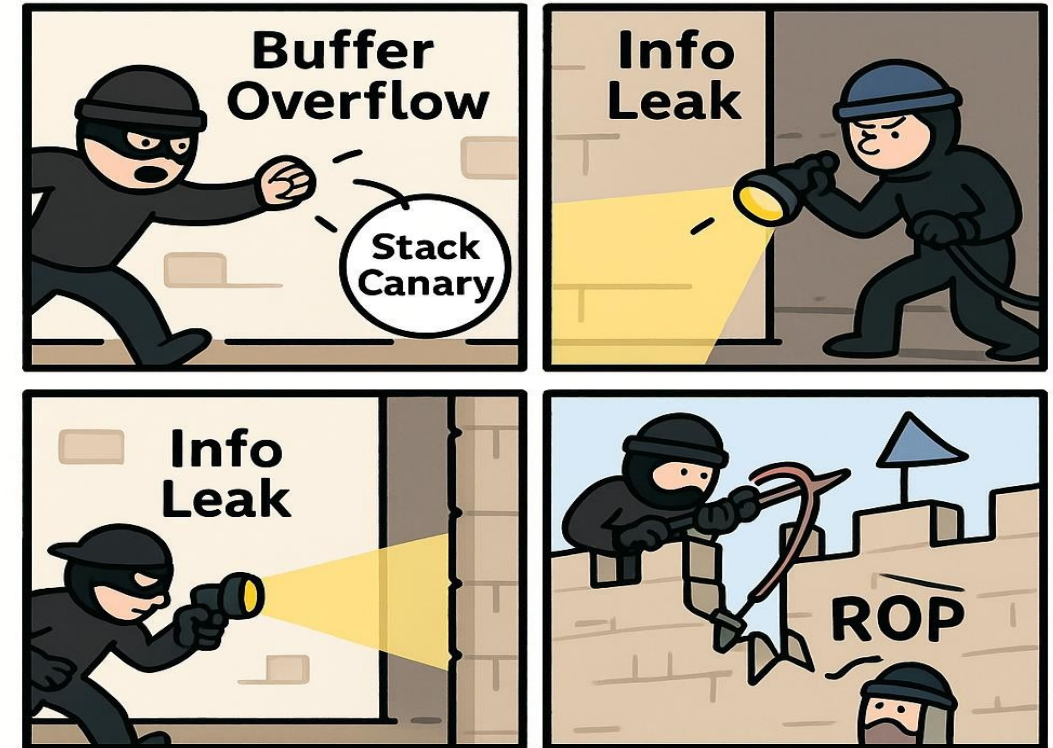
- Developers add protections: stack canaries, ASLR, DEP, and safe functions.
- The idea: *"If an attacker tries a buffer overflow, they'll crash before they succeed."*

The Attacker Strikes

- Step 1: They try a classic overflow → *blocked by a canary* (program aborts).
- Step 2: They adapt — leaking memory to **guess the canary value**.
- Step 3: They notice ASLR randomizes addresses → so they **brute force guesses** or exploit an info leak.
- Step 4: DEP stops direct code execution → they pivot to **Return-Oriented Programming (ROP)**, chaining together existing instructions.

The Reality Check

- No single countermeasure is perfect.
- Attackers evolve, defenders respond → a continuous *arms race*.
- Lesson: **Defense in depth** is key — multiple layers of security make exploitation *much harder and much costlier*.



Address Space Layout Randomization (ASLR)



- ❑ **What it does:** Randomizes the start location of key memory regions (stack, heap, libraries, etc.) each time a program runs.

- ❑ **Why it matters:**
 - Makes it **difficult for attackers to predict addresses** in memory.
 - Even if they know the vulnerable code, they can't easily guess where to jump.

- ❑ **Impact on exploits:**
 - Harder to guess the stack address.
 - Harder to guess **%ebp** (base pointer) and the address of injected malicious code.
 - Forces attackers to rely on brute force or advanced techniques (e.g., return-oriented programming).

ASLR: Counter measure



1. ASLR Disabled

(kernel.randomize_va_space=0)

- Memory addresses remain constant.
- Attackers can easily reuse the same address repeatedly.
- Exploits become **predictable**.

1

```
$ sudo sysctl -w kernel.randomize_va_space=0
kernel.randomize_va_space = 0
$ a.out
Address of buffer x (on stack): 0xbffff370
Address of buffer y (on heap) : 0x804b008
$ a.out
Address of buffer x (on stack): 0xbffff370
Address of buffer y (on heap) : 0x804b008
```

2. Partial ASLR

(kernel.randomize_va_space=1)

- Stack, heap, and shared library addresses are randomized.
- Makes guessing harder but still not impossible for determined attackers.
- **Security increases, but vulnerabilities remain.**

2

```
$ sudo sysctl -w kernel.randomize_va_space=1
kernel.randomize_va_space = 1
$ a.out
Address of buffer x (on stack): 0xbf9deb10
Address of buffer y (on heap) : 0x804b008
$ a.out
Address of buffer x (on stack): 0xbf8c49d0
Address of buffer y (on heap) : 0x804b008
```

3. Full ASLR

(kernel.randomize_va_space=2)

- Every memory run generates **new, unpredictable addresses**.
- Extremely difficult for attackers to guess the right return address.
- **Strongest protection mode** against buffer overflow exploits.

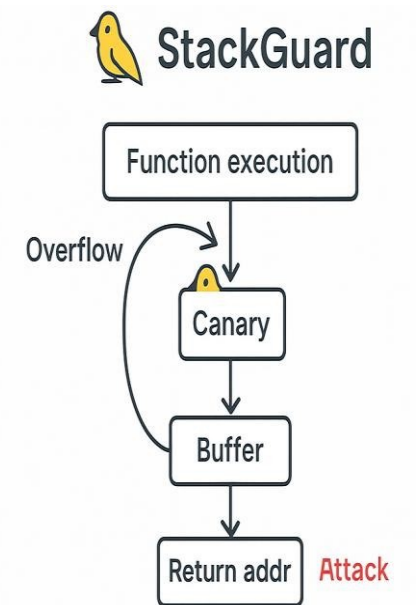
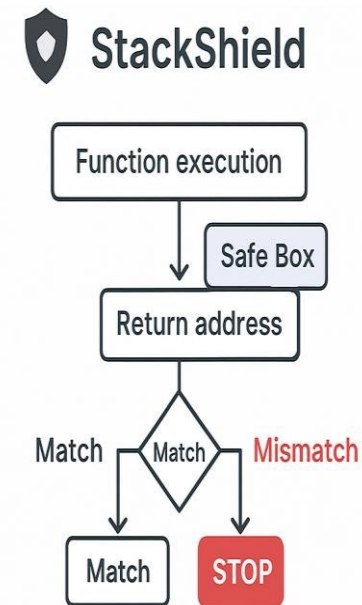
3

```
$ sudo sysctl -w kernel.randomize_va_space=2
kernel.randomize_va_space = 2
$ a.out
Address of buffer x (on stack): 0xbf9c76f0
Address of buffer y (on heap) : 0x87e6008
$ a.out
Address of buffer x (on stack): 0xbfe69700
Address of buffer y (on heap) : 0xa020008
```

StackShield vs StackGuard: What's the Difference?



Feature	StackShield	StackGuard (Canary)
Defense Mechanism	Copies return address to a safe, protected location	Inserts a "canary" value before return address on the stack
Detection	Ignores little tampering—uses stored address	Detects any overwrite of the canary before returning
Coverage	Specifically protects return addresses	Protects return address and, depending on implementation, saved frame pointer
Ease of Bypass	Can be bypassed through advanced attacks like ROP	Also vulnerable via ROP or clever overwrite without corrupting canary



💡 Extra Reading: <https://purplesec.us/learn/prevent-buffer-overflow-attack/>

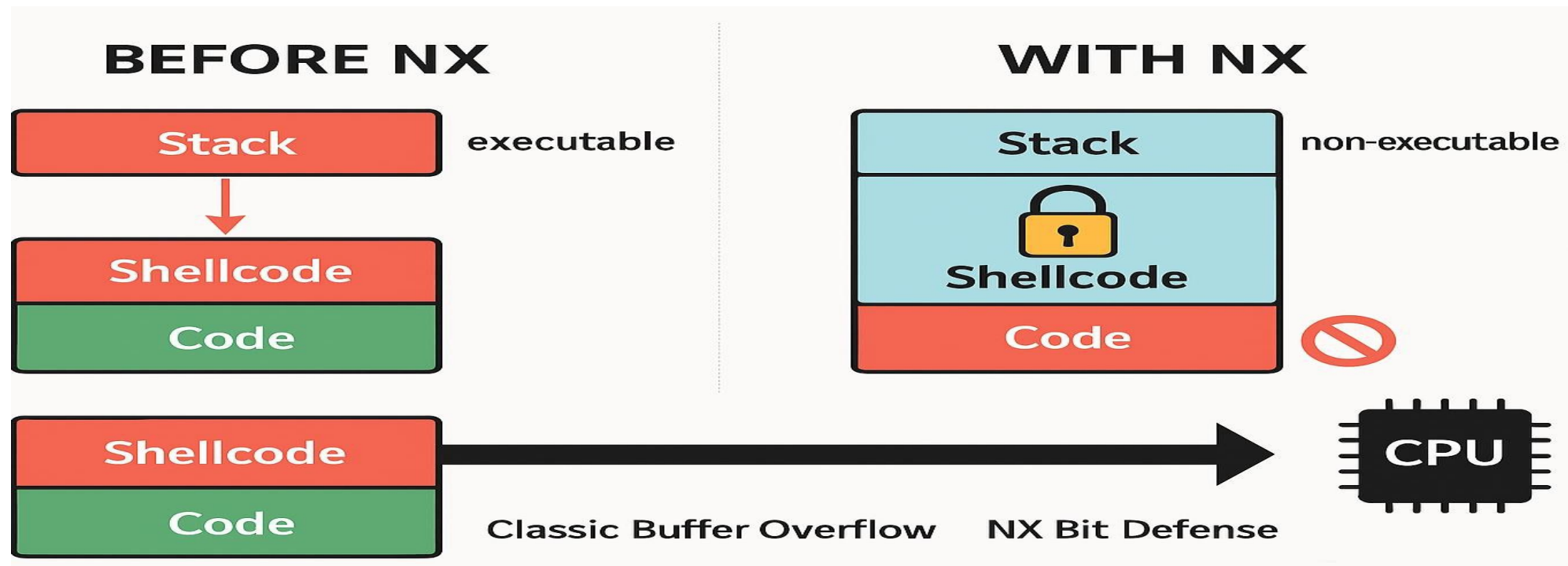
Non-Executable Stack (NX Bit Defense)



- NX bit (*No-eXecute bit*) is a CPU-level feature.
- Separates *data* from *code*: marks certain memory regions (like the stack or heap) as **non-executable**.
- Goal: Prevents injected malicious shellcode from executing on the stack.

Why it matters:

- Stops the classic buffer overflow exploit where attackers inject and run their own code directly from the stack.
- Became a **default defense** in modern CPUs and operating systems.



Memory Mapping using mmap() and Race condition



mmap() is a **system call** that maps a file or device into a process's virtual memory.

- Instead of reading/writing via file I/O functions, programs can **access file content directly in memory**.
- This improves performance but, also opens doors for **race condition vulnerabilities**.

Purpose (Why use mmap):

- **Performance:** Bypass explicit read/write syscalls; the OS handles paging, so file bytes are accessed like memory → faster I/O for large files.
- **Sharing:** Multiple processes can map the same file pages, enabling efficient inter-process data sharing
- **Convenience:** Lets code treat file contents as an array in memory, simplifying parsing and random access.

Types (mmap)

- MAP_SHARED: Changes are shared, MAP_PRIVATE: Changes are not visible to others;

Race condition (why mmap + COW can be risky):

- **What it is:** A timing bug where two kernel actions interleave in the wrong order, changing the security outcome. **Where it appears here:** A write to a MAP_PRIVATE page (triggers Copy-On-Write) overlaps with a mapping update or a memory hint (e.g., `madvise(MADV_DONTNEED)`).
- **Why it matters:** If the kernel copies/updates out of order, the write may hit the original (should-be read-only) file page instead of a private copy. (Historical case: Dirty-COW, now patched.)

Dirty-COW (CVE-2016-5195): What Happened & Why It Matterred



Core idea

A race condition in Linux's COW mechanism let a local user modify read-only files → privilege escalation (root).

How the bug was triggered:

By rapidly combining file mmap, `madvise(MADV_DONTNEED)`, and writes, an attacker could “win” the timing race so the kernel wrote to the original read-only mapping instead of a private copy.

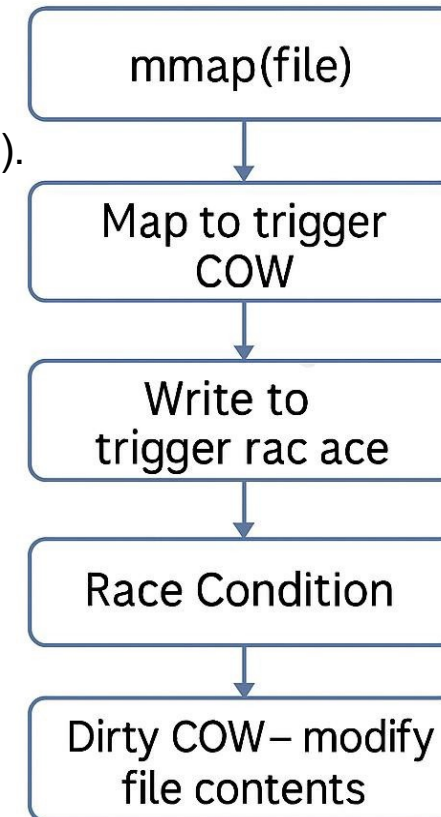
Memory Mapping using mmap() and Race Condition

Attackers do the following:

1. **Change page table entries** → force virtual memory back to the original memory (1).
2. **Use `madvise(MADV_DONTNEED)`** → discard private copy (2).
3. **Write during race window** → overwrite protected memory (1).

Result

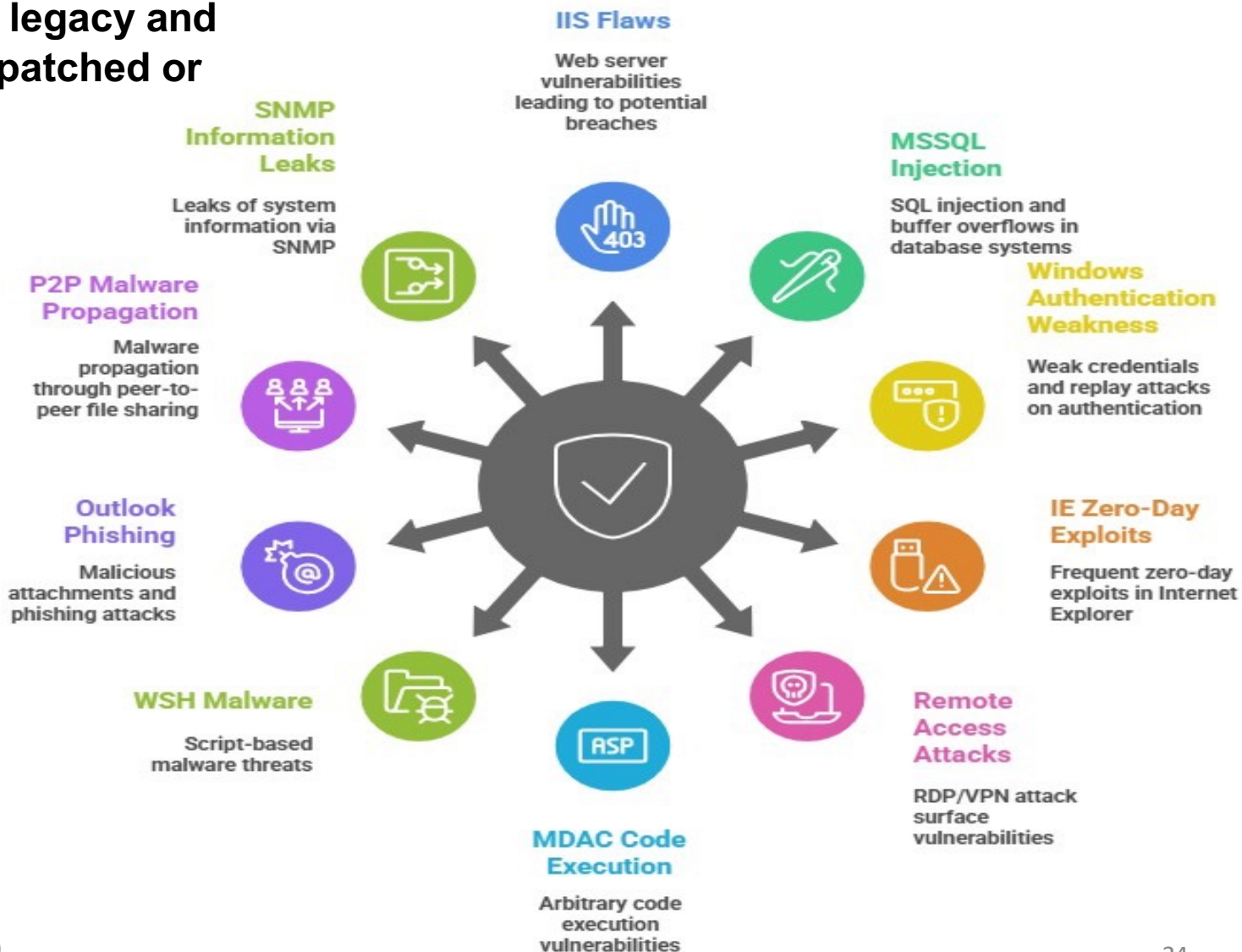
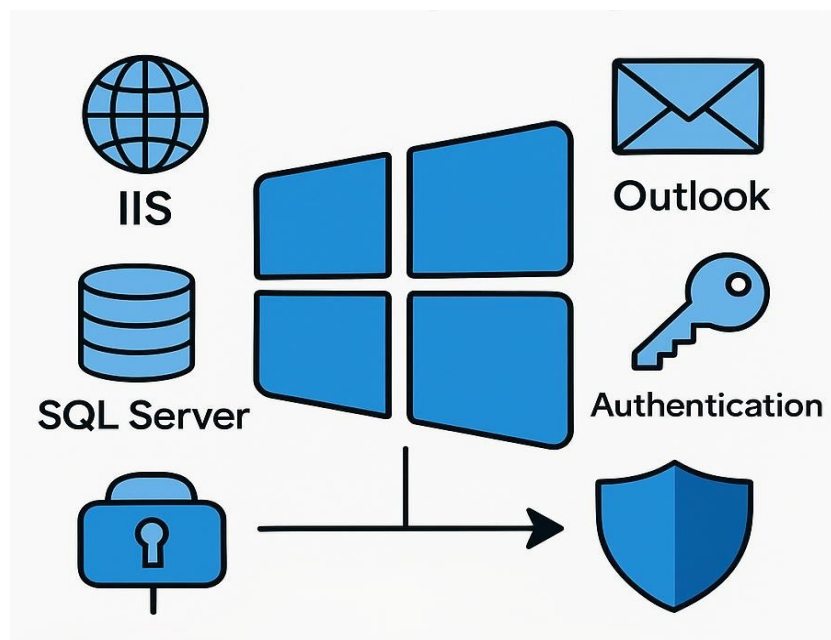
- Attack bypasses read-only protections.
- System files (e.g., `/etc/passwd`) can be overwritten.
- Privilege escalation → attacker gains **root access**.



Vulnerabilities of Desktop OS



Windows systems, as the most widely deployed desktop OS, are frequent targets of attackers. Many legacy and modern components introduce risks if not patched or secured properly.



Most Popular Mobile OS 1999 - 2026

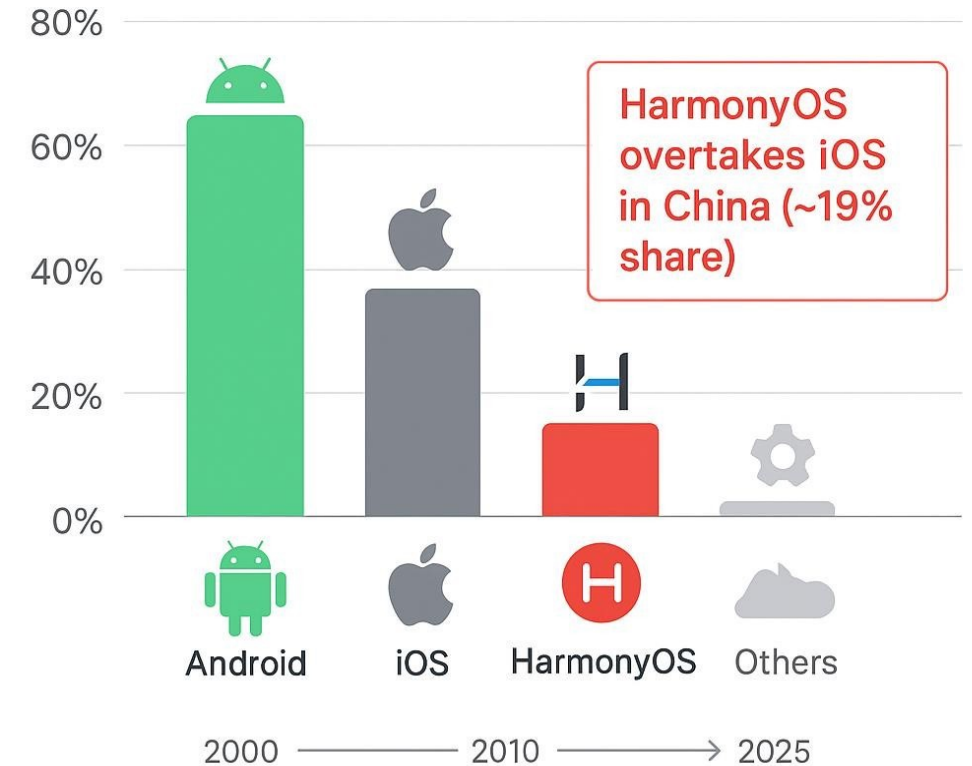


Global Market Share (2026)

- **Android:** ~68.24%
- **iOS:** ~31.48%
- **HarmonyOS** (by Huawei): ~5% globally (gaining ground in China)

China's Market (2024–25)

- **HarmonyOS:** ~19% — surpassing iOS domestically



Note: Mobile OS landscape has shifted dramatically — from Symblan & BlackBerry (2000s) → Android, iOS, HarmonyOS (205).

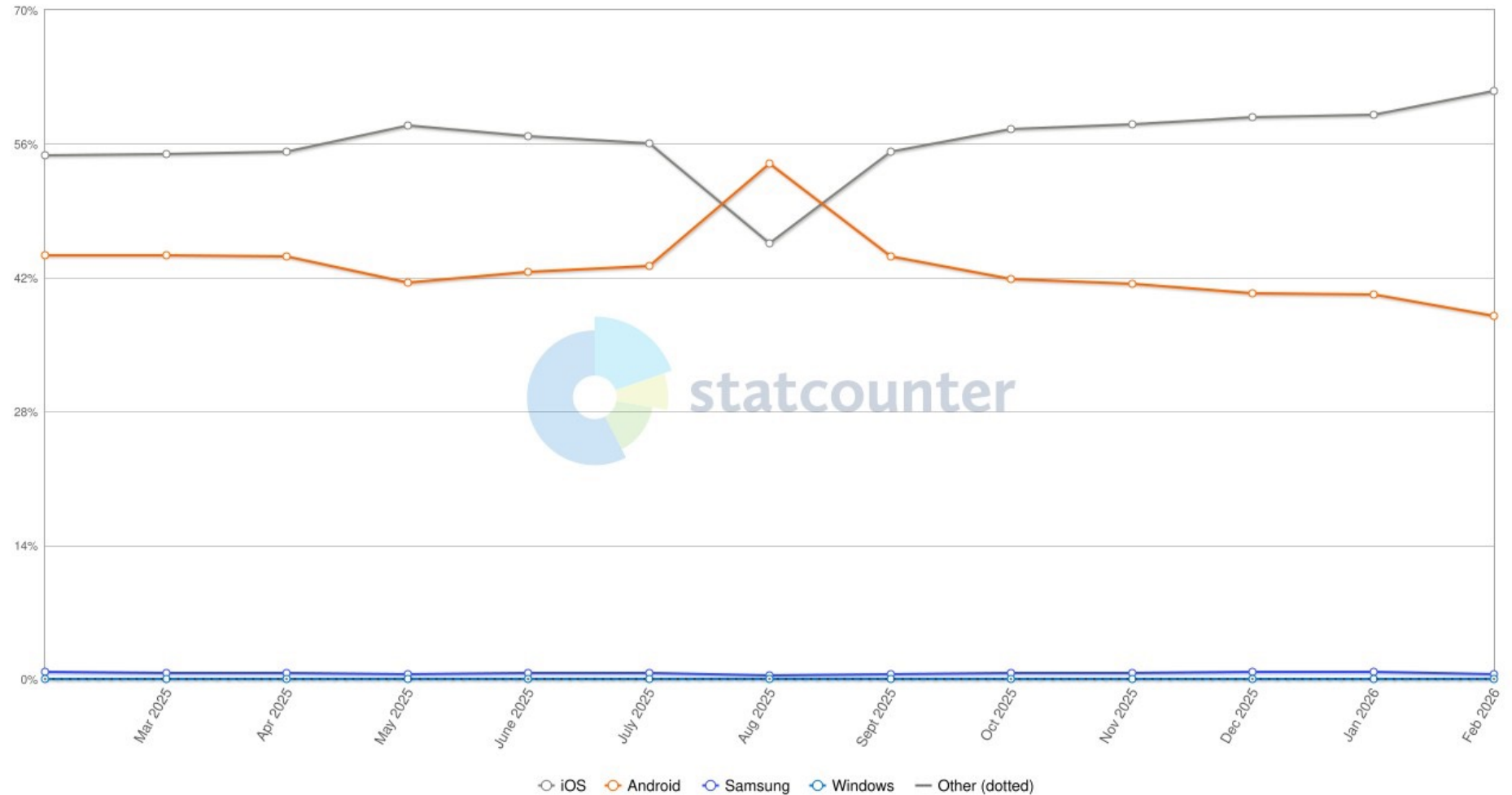
<https://www.redsearch.com.au/resources/ios-android-mobile-operating-system-statistics-australia/>
<https://izaiahnoor.pages.dev/izzhhjws-mobile-os-market-share-2025-nnrwts/>
<https://gs.statcounter.com/os-market-share/mobile/worldwide>

Australian Smart Phone Market



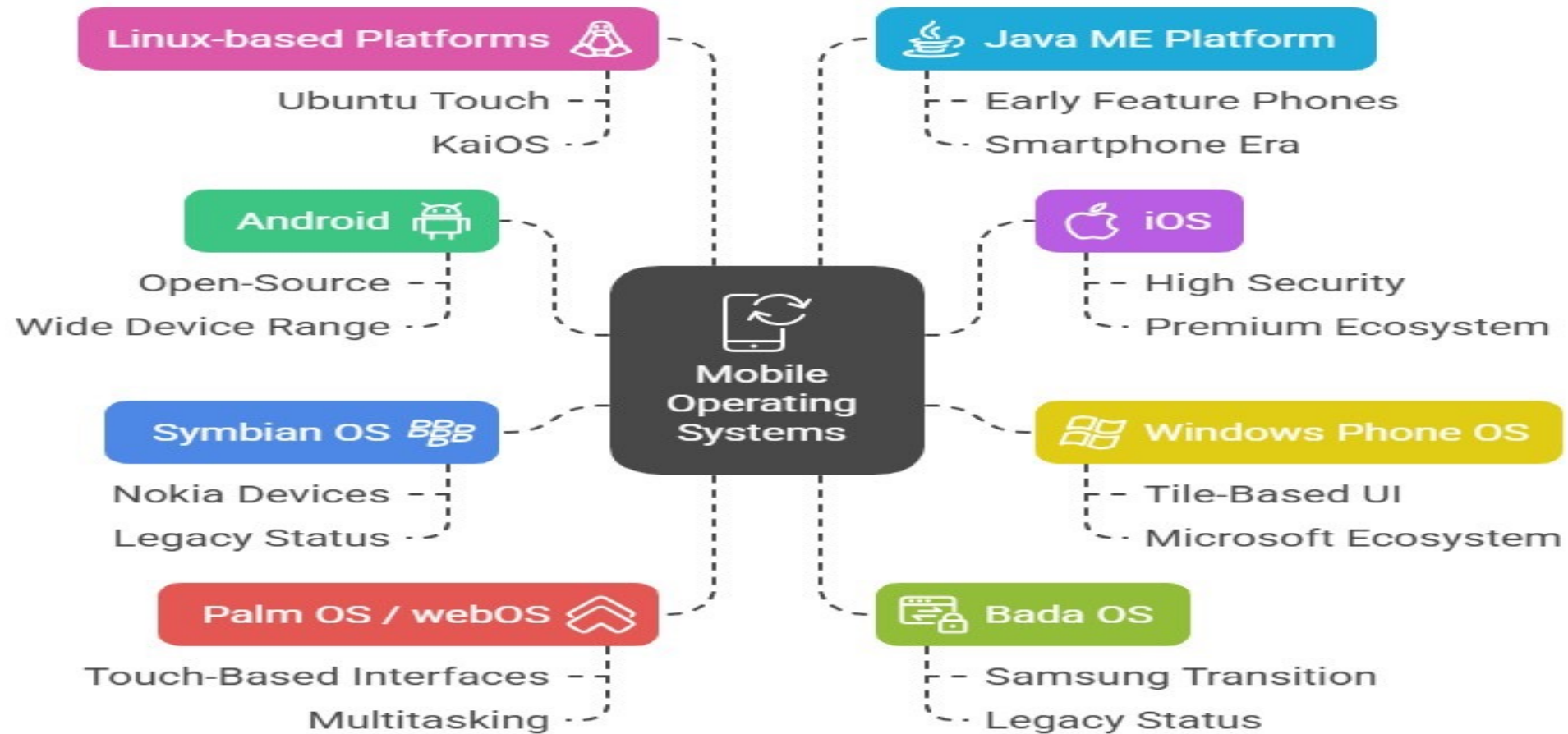
Mobile Operating System Market Share in Australia - February 2026

- **iOS:**~ 61.45%
- **Android:** ~38%
- Samsung:~0.53%
- Windows:~0.01%
- Unknown:~0.01%
- Linux~:0%



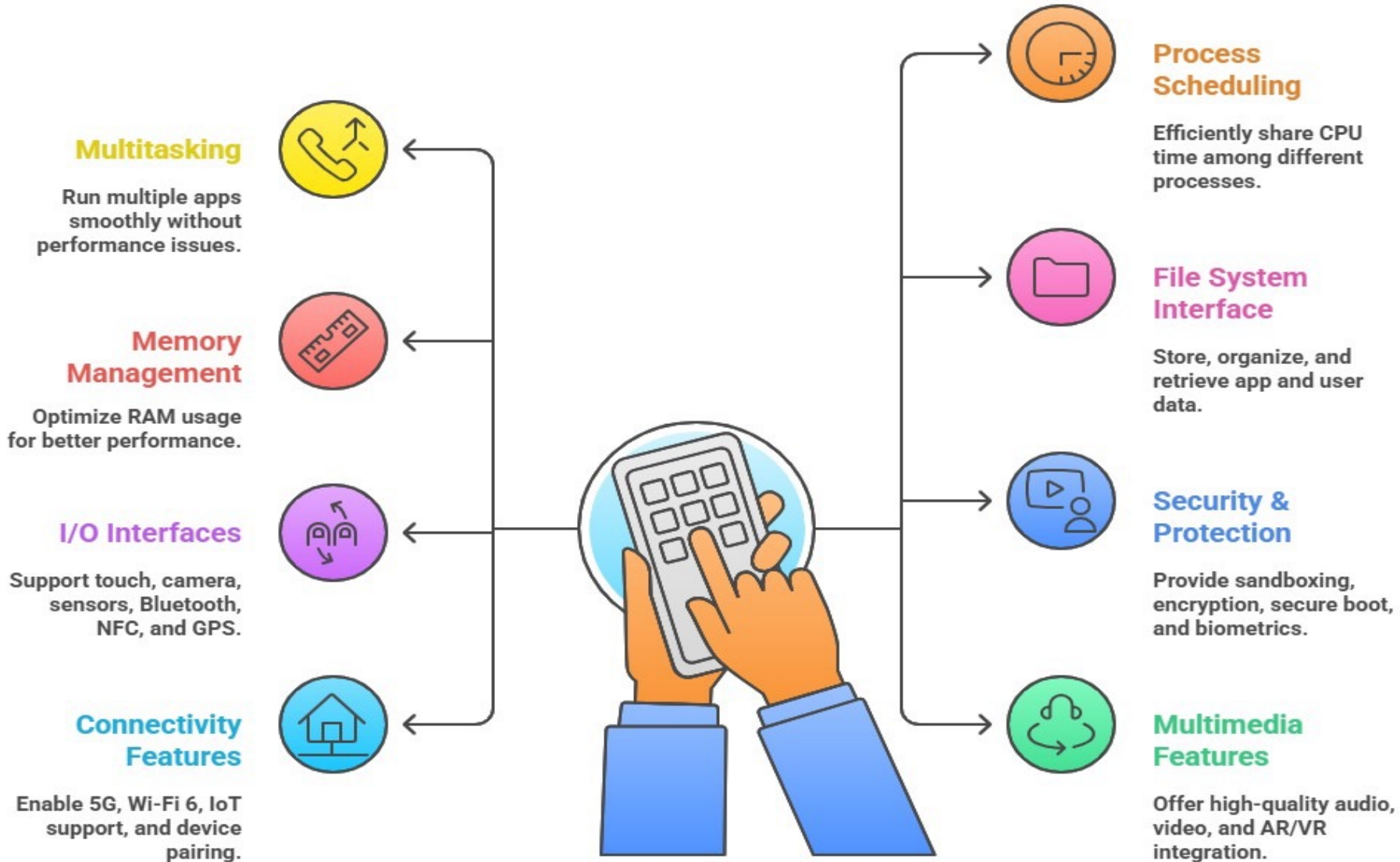
<https://gs.statcounter.com/os-market-share/mobile/australia>

Evolution of Mobile Operating Systems (History)



A Mobile Operating System (OS) is the software platform that allows applications (apps) to run on smartphones, tablets, and other portable devices. It manages hardware resources, enables connectivity, and provides a user interface.

Core Features of Modern Mobile OS



Android and iOS Architecture



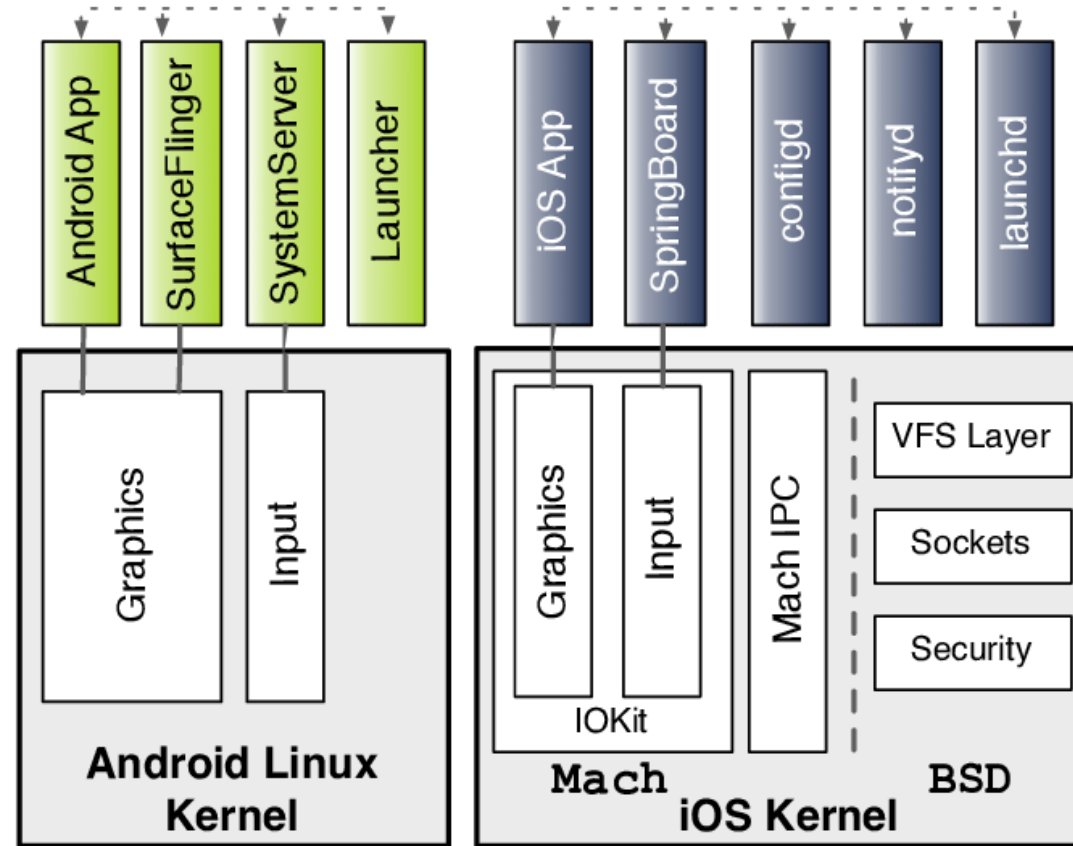
Android Architecture (Linux-based)

- **Kernel:** Built on the Linux kernel, providing low-level services (device drivers, memory, process management).

- **Libraries & Runtime:** Includes Android Runtime (ART), core libraries, and native libraries (SQLite, SSL, etc.).

- **Application Framework:** System services like SystemServer, SurfaceFlinger, and activity management.

- **Applications:** User apps + core apps (launcher, settings, dialer).



iOS Architecture (Mach + BSD-based)

- **Kernel:** Hybrid of Mach kernel + BSD components (sockets, VFS, security).

- **Core Services:** IOKit, Mach IPC, and essential system daemons (configd, notifyd, launchd).

- **Frameworks:** Cocoa Touch, UIKit, Core Foundation providing app APIs.

- **Applications:** iOS apps managed by SpringBoard (home screen manager).

Key Differences

- **Kernel Base:** Android uses **Linux**, iOS uses a **Mach/BSD hybrid**.
- **UI Management:** Android → SurfaceFlinger; iOS → SpringBoard.
- **App Lifecycle:** Android apps rely on activities & intents; iOS apps use the UIApplication model.
- **Ecosystem:** Android is **open-source & customizable**, iOS is **closed & tightly controlled**.

Jeremy Andrus et al., "Cider: Native Execution of iOS Apps on Android"

Some facts about mobile security



<https://www.le-vpn.com/security-challenges-on-mobile-devices/>

Cyber Security, OSS Lecture 5

Some facts about mobile security



5 Myths of Modern Mobile Security



Are your employees re-using work credentials on their mobile devices and possibly storing them in their browsers?



Are they using 2FA from their phones?



Are they accessing your intellectual property from their mobile devices either through email, Slack, cloud data stores, or other sources?



Are they sending sensitive texts about your organization via Text Message, WhatsApp, Signal, or Telegram? Could this data be used for extortion purposes?



Are they using their mobile devices for banking and sending sensitive content to friends and loved ones? Do you think this is their problem and not yours?

<https://verify.io/blog/5-myths-of-modern-mobile-security>

Common Security Vulnerabilities (Part 1)



❑ Insecure Data Storage

- User credentials, tokens, or cryptographic keys stored without encryption.
- Attackers exploit physical access, backups, or malware.

❑ Weak Authentication & Authorization

- Weak passwords, missing MFA, or poorly designed session management.
- Unauthorized access to sensitive apps or data.

❑ Unsecured Communication Channels

- No TLS/SSL, outdated protocols, or certificate pinning bypasses.
- Enables **Man-in-the-Middle (MITM)** attacks and eavesdropping.

❑ Device Misconfiguration

- Default passwords, disabled screen locks, or unsecured Wi-Fi.
- Leaves entry points for attackers.



Common Security Vulnerabilities (Part 2)



❑ Outdated Software & Patching

- Unpatched OS/app versions → known exploits remain usable.
- Example: Pegasus, Stagefright, and Log4j exploited via old libraries.

❑ Insecure APIs

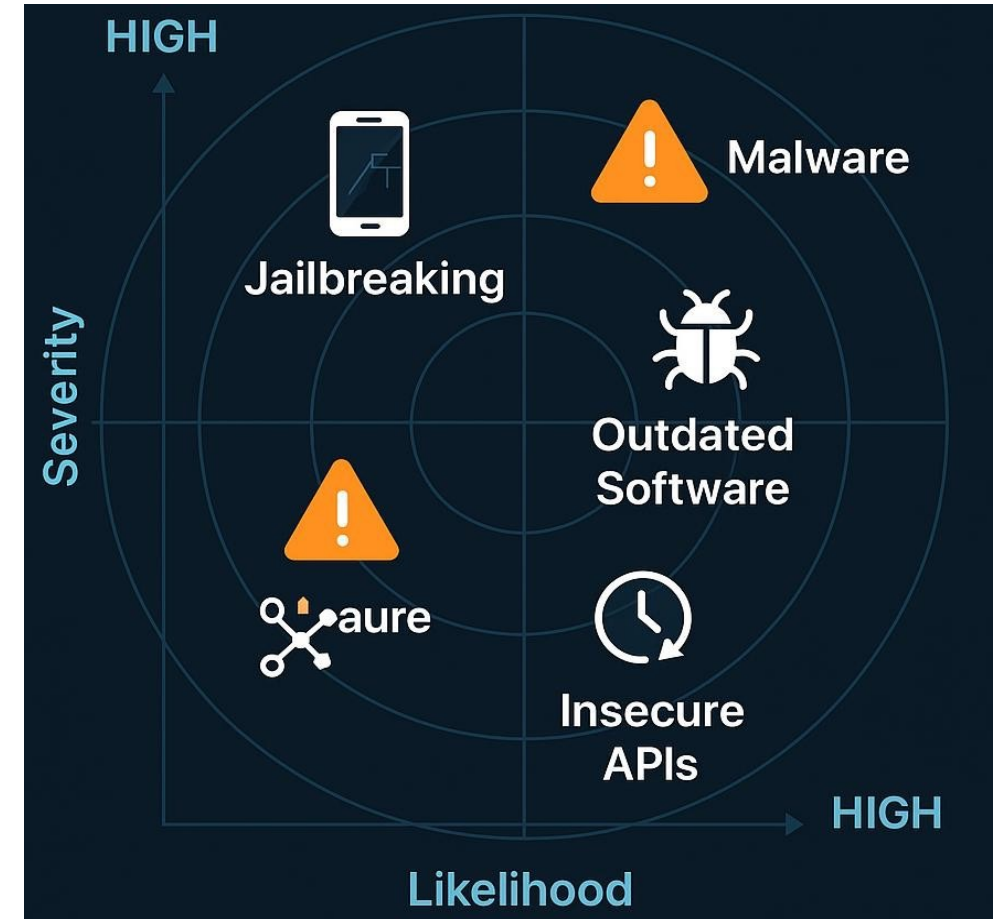
- APIs without authentication or input validation.
- Attackers hijack requests or extract sensitive data.

❑ Malware & Malicious Apps

- Fake apps in third-party stores (and occasionally official stores).
- Can steal data, record keystrokes, or lock devices (ransomware).

❑ Jailbreaking & Rooting

- Bypasses OS protections, removes sandboxing.
- Enables privilege escalation, spyware, and hidden persistent malware.



Modern & Emerging Threats



❑ Supply Chain Attacks

- Compromised SDKs, plugins, or app store libraries.
- Attackers target developers directly.

❑ Phishing via Mobile Apps

- Fake logins inside messaging/social apps.
- Example: MFA-bypass phishing kits spreading on Telegram.

❑ Spyware & Stalkerware

- State-sponsored spyware (e.g., Predator, Pegasus).
- Hidden monitoring apps targeting individuals.

❑ 5G & IoT Device Risks

- Mobile devices as **gateways into smart homes, cars, wearables**.
- Increases attack surface beyond the phone.



Security Measures (Data & Authentication)



❑ Encryption:

All sensitive data (files, credentials, tokens) must be encrypted with *modern standards* (AES-256, ChaCha20). Full-disk and file-level encryption should be mandatory.

❑ Secure Storage APIs:

Use platform-native secure storage (iOS Keychain, Android Keystore, Samsung Knox). Avoid storing secrets in shared prefs, files, or databases.

❑ Multi-Factor Authentication (MFA):

Protect against credential theft by combining *something you know* (password), *something you have* (device/OTP), and *something you are* (biometrics).

❑ Password Hygiene:

Enforce policies like minimum 12 characters, no reuse, and periodic checks against known-breached password lists (e.g., HaveIBeenPwned integration).

✓ *Modern Twist:* Breaches today often start with weak credentials; these controls make brute force and credential stuffing much harder.



Security Measures (Protocols & Communications)



❑ Secure Authentication Protocols:

Always rely on OAuth 2.0 / OpenID Connect. Never roll your own authentication.

❑ TLS/SSL (with best practices):

Use TLS 1.3 only, disable weak ciphers, enforce HSTS. Protect against downgrade attacks.

❑ Certificate Pinning:

Ensure apps trust *only* the expected server certificates, defending against forged/malicious certs.

❑ End-to-End Encryption (E2EE):

For chat/messaging apps, ensure only endpoints hold the decryption keys. Even service providers should not decrypt.

✓ *Modern Twist:* In 2025, regulators are scrutinizing encryption backdoors — strong TLS and E2EE are critical for compliance and trust.



Security Measures (Device & API Hardening)



❑ API Security:

Lock APIs with OAuth 2.0, JWT tokens, rate limiting, and input validation. Expose *least privilege* only.

❑ Strong Device Locks:

Encourage biometrics (FaceID, fingerprint) and fallback PINs. Enforce enterprise policies that require device lock before app use.

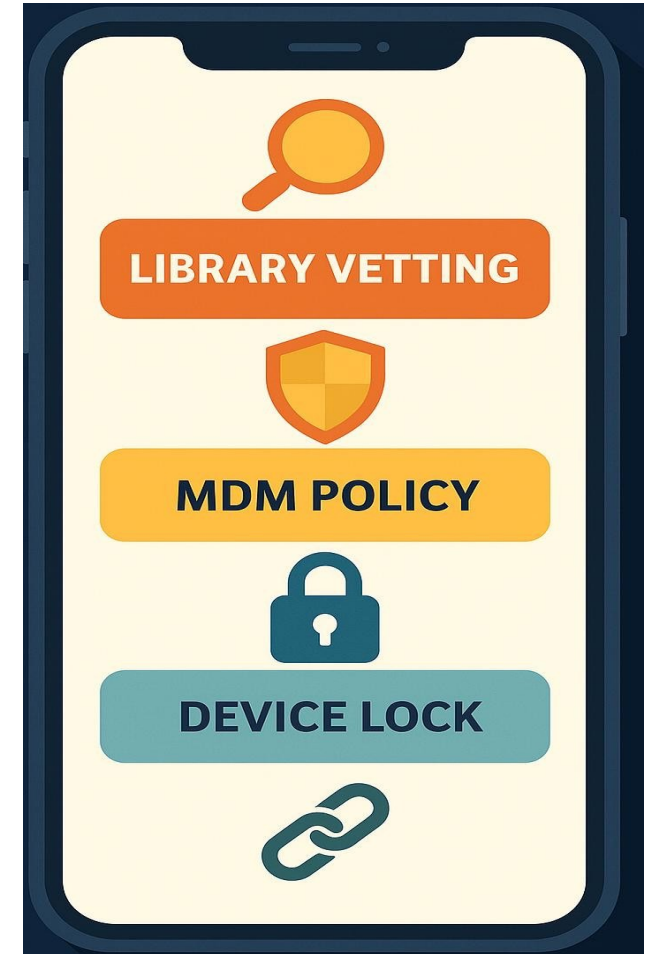
❑ Mobile Device Management (MDM):

Enterprises must use MDM (e.g., Intune, VMware Workspace ONE) to enforce encryption, VPN, remote wipe, and compliance monitoring.

❑ Library & SDK Vetting:

Check all third-party code for CVEs before integration. Supply-chain attacks via libraries are one of the biggest threats (e.g., malicious NPM/PyPI packages).

✓ *Modern Twist:* 2024–2025 has seen a surge in **supply chain attacks** targeting mobile SDKs — vetting is no longer optional.



Penetration Testing on Android OS

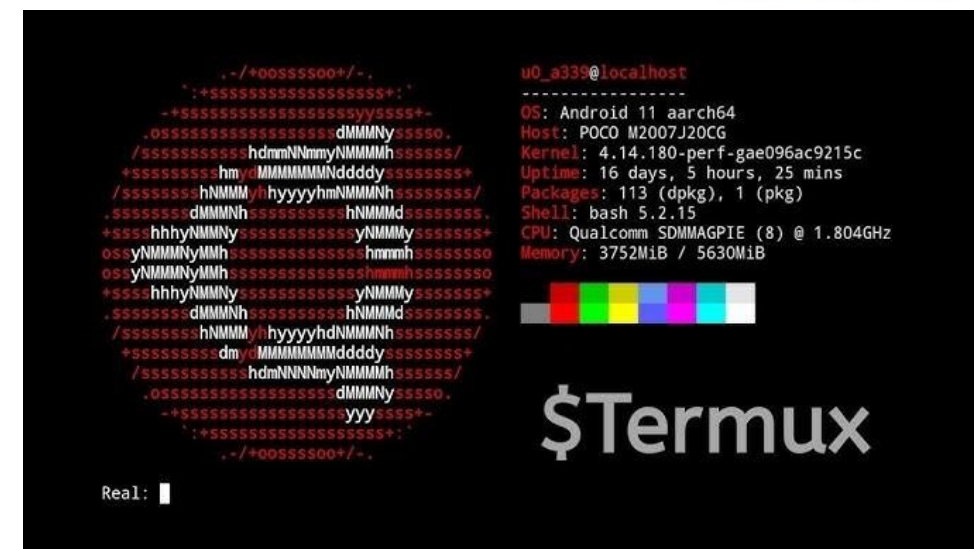


What is Termux?

- A **powerful terminal emulator** for Android.
- Provides a **full Linux environment** directly on Android devices.
- Widely used by **penetration testers, security researchers, and developers**.

Key Features:

- Access to a wide range of Linux tools & utilities.
- Supports **APT package management** (similar to Debian-based systems).
- Direct **integration with Android file system**.
- Useful for **network scanning, password cracking, reverse engineering, and scripting**.



<https://github.com/termux>

Why Termux for Security Testing?

- Portable and lightweight (no rooting required).
- Allows running **Metasploit, Nmap, Hydra, Nikto**, and many other tools.
- Excellent for **hands-on cybersecurity learning and mobile pentesting labs**.

Popular Tools Available in Termux:



- **Nmap** – Network scanning, host discovery, and port enumeration.
- **Metasploit Framework** – Exploit development and penetration testing automation.
- **Hydra** – Fast password cracking (supports many protocols like SSH, FTP, HTTP).
- **Nikto** – Web server vulnerability scanner.
- **SQLmap** – Automated SQL injection tool for database penetration testing.
- **Wireshark** → Network traffic analysis & packet sniffing.
- **Aircrack-ng** – Wireless network auditing (Wi-Fi password testing).
- **John the Ripper** – Password hash cracking.

Why Use Termux for These Tools?

- Portable & runs directly on Android.
- No rooting required for most tools.
- Lightweight but powerful for real-world **ethical hacking** tasks.
- Great for **learning cybersecurity hands-on** on-the-go.
- ✓ **Tip:** Combine tools for multi-step testing (e.g., Nmap → SQLmap → Metasploit).



<https://github.com/termux>



What is iSH Shell ?

- A lightweight tool for bringing Unix-like capabilities to iOS.
- Enables penetration testing without requiring jailbreaking.
- Open-source project that emulates a Linux shell environment on iOS devices.
- Provides a portable and secure environment for security researchers.

Features:

- Linux Environment Emulation (Alpine Linux based).
- Package Management (apk).
- Command-Line Tools (grep, curl, ssh, etc.).
- Scripting Capabilities for automation.
- Portability and non-intrusive deployment on iOS.



For more information:

<https://ish.app/>

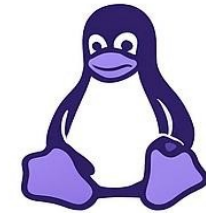
Penetration Testing on iOS



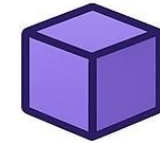
iSH Shell allows installation of well-known penetration testing tools on iOS:

- **Nmap** → Network scanning & discovery.
- **Curl** → Interacting with APIs & web services.
- **Git** → Cloning repos with security/pen-testing scripts.
- **Netcat** → Networking utility for reading/writing raw data.
- **SQLmap** → Automated SQL injection & DB exploitation.
- **Hydra** → Brute force login/password cracker.
- **Nikto** → Web server vulnerability scanner.
- **TCPdump** → Packet capture & analysis.

<https://ish.app/>



Linux
emulation



Package
management



Terminal
CLI tools



Scripting
automation



Mobile
portability

Additional and Advanced Android Pentesting Platforms & Tools



1. Kali NetHunter

- A free, open-source mobile penetration testing platform based on **Kali Linux**, designed for Android devices.
- Available in Rooted, Lite, and **Rootless** versions—works even without root on many devices.
- Comes with capabilities like **USB HID attacks, Wi-Fi frame injection, EvilAccess Point (MANA), BadUSB MITM.**
<https://www.seanobi.com/2024/12/kali-nethunter-vs-its-alternatives.html>

2. LinuxDroid

A security toolkit that brings a Linux environment (Ubuntu or Kali) onto Android without needing root.
Supports powerful tools like **Nmap, Metasploit, Wireshark**, all run within a Linux chroot.
<https://github.com/AryanVBW/LinuxDroid>

3. MobSF (Mobile Security Framework)

A comprehensive, **all-in-one platform** for mobile application assessment (Android & iOS).
Features include **static analysis, dynamic analysis**, and **API testing**.
<https://github.com/ajinabraham/Mobile-Security-Framework-MobSF>

4. zANTI & Astra Pentest

- **zANTI**: A mobile penetration testing suite for network auditing and vulnerability scanning.
- **Astra Pentest**: A hybrid pentesting platform with automated scans, domain-specific checks (OWASP Mobile Top 10), and expert human review.
<https://www.getastra.com/blog/mobile/android/android-pentesting-tools/#zanti>

5. Frida, Objection, and Radare

Frida: A dynamic instrumentation toolkit used to hook into app processes at runtime—ideal for modifying and testing.
Objection: A runtime mobile security framework built on Frida for bypassing root/jailbreak detection.
Radare / R2frida: Reverse engineering frameworks for deep analysis of binaries, now often combined with Frida for dynamic insights.
<https://github.com/tanprathan/MobileApp-Pentest-Cheatsheet>

6. Appie, Android Tamer, Mobexler

Full-fledged environments for mobile app testing:
<https://www.getastra.com/blog/mobile/mobile-app-pentesting-tools/>

AND MUCH MORE LOOK FOR MORE
<https://github.com/DamnYatin/All-in-one-termux-tools>

Summary



- OS security is one of the most important parts of Cybersecurity
- Every action dealing with OS must be done carefully
- Buffer overflow attack is one of the dangerous attack and is a common security flaw
 - Attacker could exploit buffer overflow to run injected code
- Various tools are being used to analyse further investigations on OS security
- Termux and iSH Shell can both be utilized as tools for penetration testing on mobile operating systems